ADVANCED PHISHING DETECTION USING MULTI-MODAL ARCHITECTURE

MALINGA SAMARAKOON

Submitted in partial fulfillment of the requirements of the degree of

B.Sc. Honours in Computer Science

Department of Statistics and Computer Science

University of Peradeniya

ABSTRACT

Phishing remains one of the most pervasive and costly cyber threats, with attackers continuously evolving their tactics through sophisticated social engineering, content obfuscation, and multi-channel attack vectors that circumvent traditional filter-based defenses. Despite significant advances in machine learning for security, existing phishing detection systems face critical limitations: lack of transparency hindering analyst verification and regulatory compliance, vulnerability to adversarial evasion attacks, computational overhead unsuitable for client-side deployment, and single-modality focus ignoring complementary signals across different data types.

This research presents a novel lightweight, multi-agent phishing detection architecture specifically designed to address these fundamental gaps through integrated solutions for accuracy, explainability, robustness, and edge deployability. The system uniquely combines four specialized detection agents such as **Text Agent** (analyzing email and webpage content), **URL Agent** (examining link characteristics), **Metadata Agent** (assessing header anomalies and sender reputation), and **Image Agent** (detecting visual deception). Each trained on domain-relevant features using compact, optimized models. Text, URL, and Metadata agents utilize **DistilBERT**-based classifiers fine-tuned on large-scale public datasets including CEAS (8,951 emails), Enron (35,184 emails), SpamAssassin (7,951 emails), and PhiUSIIL (235,795 URLs). The Image agent employs **MobileNetV2**, a lightweight convolutional neural network (3.5M parameters), trained on approximately 2000 website screenshots and email images.

Individual agents demonstrated exceptional performance on their specialized domains: Text agent achieved F1-score of **0.9932** with 99.32% accuracy, URL agent scored **0.9902** F1-score with 99.85% accuracy, and Metadata agent attained **0.9856** F1-score with 95.75% accuracy. Conversely, the Image agent remained highly limited—despite augmentation and careful design, it suffered from persistent class imbalance, data scarcity, and low recall for phishing visuals (ROC-AUC=0.59, F1 $\ll$ 1 on real-world test cases). These results highlight widely reported challenges in visual phishing detection and the need for larger, more diverse image corpora. Unlike traditional ensemble stacking methods, an **XGBoost (Extreme Gradient Boosting) meta-classifier** intelligently fuses agent outputs through learned importance weights, achieving a final **system accuracy of 94.76%** with **AUC of 0.93**, effectively capturing non-linear agent interactions and prioritizing most reliable signals in ambiguous cases.

The system incorporates two critical innovations addressing the explainability, performance and trade-off: **SHAP (SHapley Additive exPlanations)** integration provides theoretically grounded, human-interpretable attribution of predictions to specific agent contributions and features, enabling operational transparency without sacrificing accuracy. Comprehensive **adversarial training protocols** enhance robustness against sophisticated evasion attacks (homoglyph domains, character substitution, obfuscation), improving accuracy retention under perturbation from 63% to 93% while maintaining 99.18% clean accuracy.

This research demonstrates that notable accuracy phishing detection (**94.76% accuracy**), model interpretability (**SHAP-based explanations**), adversarial robust-

ness (**92.87% accuracy on perturbed inputs**), and edge deployability (**<50ms latency**) can be simultaneously achieved through careful architectural design, comprehensive adversarial training, and systematic optimization. The system balances precision (0.91 for phishing class), recall (0.97 for phishing class), and computational efficiency, providing a production-ready foundation for trustworthy, scalable phishing protection deployable on mobile devices, browser extensions, and edge gateways.

Keywords: Phishing Detection, Multi-Modal Learning, DistilBERT, Explainable AI, SHAP, Adversarial Robustness, XGBoost Ensemble, Edge Computing, MobileNetV2, Model Interpretability

DECLARATION

I hereby declare that except where specific reference is made to the work of others, the contents of this report are original and have not been submitted in whole or in part for consideration for any other degree or qualification in this, or any other university. This dissertation is my own work and contains nothing which is the outcome of work done in collaboration with others, except as specified in the text and Acknowledgements. This dissertation contains fewer than 65,000 words including appendices, bibliography, footnotes, tables and equations and has fewer than 150 figures.

.....
Malinga Samarakoon
November 25, 2025

.....
Prof. Saluka R. Kodituwakku
November 25, 2025

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Prof. Saluka Kodituwakku**, Senior Professor of the Department of Statistics and Computer Science, and my research supervisor, for his continuous guidance, valuable advice, and insightful suggestions throughout the course of my research. His expertise and encouragement were instrumental in shaping this study.

My heartfelt appreciation goes to **Dr. Ruwan Nawarathna**, Senior Lecturer of the Department of Statistics and Computer Science, who served as the course coordinator and provided invaluable guidance, encouragement, and feedback during the research process. His mentorship played a crucial role in the development of this study.

I am deeply grateful to **Dr. Hakim A. Usoof**, Senior Lecturer and Head of the Department of Statistics and Computer Science, for granting me the opportunity and necessary support to undertake this research work.

I would also like to extend my gratitude to **Prof. Amalka J. Pinidiyaarachchi**, **Dr. Erunika O. Dayarathna**, and **Dr. Ruwanthini Siyambalapitiya**, Senior Lecturers of the Department of Statistics and Computer Science, for their valuable advice and continuous support in completing my research so far.

My appreciation further goes to the faculty and staff of the Department of Statistics and Computer Science, University of Peradeniya, for their constant assistance and the resources they made available to facilitate my work.

I am also thankful to my peers for their camaraderie, collaboration, and the engaging discussions that motivated me throughout my academic journey.

Lastly, I would like to express my deepest gratitude to my parents for their unwavering encouragement, love, and support throughout my studies.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF FIGURES

enumitem

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Phishing is a pervasive cyber threat in which attackers masquerade as trustworthy entities (via emails or websites) to deceive users into divulging sensitive information or installing malware (Lim et al., 2025). It remains one of the most common and damaging forms of cyberattack, accounting for a considerable share of recent security breaches. The scale of the problem continues to grow: for example, the Anti-Phishing Working Group (APWG) reported over 1,025,000 phishing attacks in a single quarter of 2024 – the highest quarterly volume on record (Rao et al., 2025). Such trends underscore the importance of effective phishing detection in today's cybersecurity landscape. Indeed, phishing detection is high-priority but highly challenging – attackers continuously refine their tactics to bypass defenses, making phishing "one of the biggest challenges" in cybersecurity due to these continuously emerging tactics. Automated detection systems are therefore essential to mitigate the "devastating results" of successful phishing attacks (Hosseinzadeh et al., 2025).

Over the years, numerous anti-phishing techniques have been developed. Early approaches relied on heuristics and blacklists of known phishing sites or emails. While blacklist-based solutions can block known malicious URLs, they fail to catch new ("zero-day") phishing sites. Heuristic methods that inspect URL lexical features, website HTML, or visual similarity have improved coverage, but each has limitations: source code analysis can be evaded by dynamically generated content, and image-based visual similarity detection is computationally expensive (Rao et al., 2025). These limitations become especially problematic on resource-constrained devices (e.g. smartphones), which often cannot afford the high latency or memory usage of such method.

In recent years, the rapid advances in machine learning (ML) and deep learning have provided new avenues for phishing detection. ML-based classifiers can learn to recognize subtle patterns in phishing emails or URLs beyond what any fixed rule set could achieve. Notably, modern deep learning models (e.g. transformer-based language models and convolutional neural networks) have achieved state-of-the-art accuracy in phishing detection tasks. For instance, transformer models like BERT and DistilBERT have demonstrated excellent effectiveness in detecting phishing content, with DistilBERT achieving exceptional accuracy and outperforming other models in phishing URL classification (Hosseinzadeh et al., 2025). Likewise, lightweight convolutional neural networks (CNNs) such as MobileNetV2 enable image-based phishing detection (e.g. analyzing webpage screenshots or email images) in resource-limited settings. Given that phishing attempts may involve multiple modalities – the URL link, the email/webpage text, and even logos or images – researchers are increasingly exploring multi-modal detection strategies.

By leveraging multiple cues (URL features, textual content, and visuals), such approaches aim to catch phishing attacks that might slip past single-feature detectors. This has led to the idea of multi-agent phishing detection systems, wherein specialized detectors (or "agents") handle different data modalities (e.g. one agent analyzes textual content with an NLP model, while another inspects images with a CNN), and their outputs are combined. Recent studies suggest that this kind of multi-agent (or multi-model) collaboration can yield more accurate and comprehensive phishing detection than any single model alone.

1.2 Problem Statement

Despite progress in phishing detection, significant gaps and limitations persist in existing systems that this research seeks to address. First, many phishing detectors operate as "black-box" models with little interpretability. Users and security analysts often cannot understand why a given email or URL was flagged as phishing, which undermines

trust in the system's alerts. This lack of explainability is increasingly problematic: not only does it reduce user confidence, but emerging regulatory frameworks (especially in finance and healthcare) are beginning to require AI decisions to be explainable. Black-box phishing detectors are thus becoming untenable in sensitive domains. Second, attackers are continually adapting their techniques from AI-generated phishing content to clever URL obfuscation – to evade detection.

Traditional filters and even many ML models struggle to keep up with these evolving threats, resulting in undetected phishing emails (false negatives) or conversely, frequent false alarms. High false positive rates (legitimate messages misclassified as phishing) disrupt business communications, while false negatives leave users exposed. This indicates a need for improved adversarial robustness and adaptability in phishing detectors. Third, existing phishing detection solutions often face practical deployment constraints. Some methods (e.g. visual similarity algorithms or large ensemble classifiers) incur high computational cost or latency (Rao et al., 2025), making them unsuitable for real-time use.

Many advanced models are too bulky to deploy on end-user devices; for example, a full BERT-based model or large CNN might be hundreds of megabytes, which is impractical for a browser plugin or mobile app. Edge compatibility – ensuring the detection system is lightweight (limiting size about 100 MB) and low-latency is a commonly unmet requirement. Furthermore, many approaches focus on a single data modality (text only or URL only or image only), limiting their effectiveness against sophisticated phishing webpages that combine deceptive text and visuals. In summary, the key problems are: **(1)** lack of explainability in phishing detection models, **(2)** vulnerability to adaptive adversarial tactics and limited generalization to new phishing variants, **(3)** deployment hurdles due to large model size or slow performance, and **(4)** incomplete analysis when using only one type of feature. These challenges motivate the present study to develop a more robust, interpretable, real-time phishing detection solution.

1.3 Research Objectives

The main objective is to develop a multi-agent phishing detection system that overcomes current limitations by integrating multiple specialized detectors (for text, URL, metadata, and images) into a unified, robust, and explainable framework.

Specific Objectives:

1. Multi-Modal Detection: Leverage a text analysis agent (DistilBERT transformer) for email/webpage content and a visual analysis agent for images (website screenshots or logos), along with URL and metadata features, and fuse their outputs to improve phishing detection accuracy across different attack vectors.

2. Adversarial Robustness: Incorporate adversarial training and defensive techniques to make the detection models more resilient against evasion tactics (e.g. perturbations in text or images designed to fool classifiers), thereby reducing false negatives on cleverly crafted attacks.

3. Explainability: Integrate SHAP (Shapley Additive Explanations) or similar eXplainable AI methods to provide human-interpretable explanations for the model's decisions – identifying which features (words, image regions, URL attributes) influenced a phishing prediction - to improve user trust and meet transparency requirements.

4. Edge Deployment Efficiency: Optimize the system for real-time performance and low resource usage, and minimal inference latency, enabling deployment on edge devices such as email clients or browser extensions without significant impact on user experience.

5. Empirical Evaluation: Evaluate the proposed system on benchmark phishing datasets (specifically, the PhiUSIIL phishing URL dataset and a large merged email phishing dataset, reduced from 300k raw emails to 120k cleaned samples) to measure its detection accuracy, false positive rate, execution speed, and ex-

planation quality. Compare performance against baseline single-modal models to quantify improvements in effectiveness and robustness.

1.4 Research Questions

To guide the investigation, the study addresses the following key research questions

Q1: : Multi-Modal Efficacy - How much can a combined multi-modal (text + URL + metadata + image) approach improve phishing detection performance compared to single-modal classifiers? Does the multi-agent system achieve higher accuracy or lower false positives when detecting phishing emails and websites across diverse attack scenarios?

Q2: : Adversarial Resilience - To what extent does adversarial training enhance the system's robustness against sophisticated phishing attempts (e.g., content obfuscation, image morphing, or adversarial perturbations)? Can the trained models maintain high detection rates on adversarially manipulated inputs that would normally evade conventional detectors?

Q3: : Explainability and Trust - How effective are the integrated SHAP-based explanations in conveying the rationale behind model predictions? Do these explanations meaningfully highlight known phishing indicators (such as suspicious keywords or image artifacts), and do they improve user/analyst trust in the system's alerts?

Q4: : Edge Performance - Can the multi-agent system operate within the constraints of edge environments (limited memory and processing power) while still providing real-time phishing detection? Specifically, does the final model meet the edge device friendly size goal and low latency requirements without significantly sacrificing detection accuracy?

1.5 Scope of the Study

This research focuses on phishing detection in the context of websites and emails, utilizing data driven deep learning methods. The scope is confined to classification of phishing vs. legitimate instances based on content and metadata; it does not encompass broader incident response or user education interventions. The system is designed and evaluated using two primary datasets: **(a)** the PhiUSIIL Phishing URL dataset, a large collection of labeled phishing and legitimate URLs with extracted features, and **(b)** a curated phishing email dataset composed of several merged sources totaling approximately 120,000 cleaned email samples (down from an initial 300,000 raw emails). These datasets provide textual content (email bodies, headers), URLs, and in some cases images (email attachments or webpage screenshots) for analysis. The study concentrates on phishing websites and email attacks, and thus excludes other social engineering vectors such as **voice phishing (vishing)**, **SMS phishing (smishing)**, or attacks on mobile application interfaces. While the proposed architecture emphasizes edge deployability, the project's scope does not extend to developing a fully finished mobile app or browser plugin; instead, the models are evaluated in a controlled setting that simulates resource constraints. The performance metrics are obtained via google colab paid playground on the chosen datasets – issues of live deployment, user interface integration, or real-time threat feed updates are outside the scope. By clearly defining these boundaries, the study ensures a feasible and focused investigation into improving phishing detection within the chosen domain.

1.6 Significance of the Study

This study holds significance on several fronts. Academically, it contributes to the state of the art in phishing detection by exploring a novel combination of techniques – a unified multi-modal system employing both Natural Language Processing (DistilBERT) and Computer Vision (ResNet/MobileNetV2), augmented with explainability and adversarial defense. Most of the prior research has often examined these aspects

in isolation; for example, some works focus on text-based phishing email classification, while others explore image-based website verification or XAI methods. By integrating multiple approaches into one framework, the research provides insight into how these components can synergize to address each other's weaknesses. Notably, demonstrating high detection performance and meaningful explainability together would be a valuable contribution, since it bridges the gap between raw efficacy and practical trustworthiness in security AI. The findings can inform future academic developments of robust, transparent cyber defense models.

From an industrial and practical standpoint, the proposed system addresses pressing needs of cybersecurity operations. Phishing continues to be a leading cause of security breaches and financial losses for organizations. An improved detection system that yields fewer false positives will reduce alert fatigue for security teams and minimize disruption of legitimate business emails. At the same time, better false-negative performance (through multi-modal cues and adversarial training) directly translates to preventing more phishing incidents, thereby averting potential data breaches. The incorporation of SHAP explainability is significant for industry adoption: it can help security analysts quickly verify why an email was flagged (e.g. highlighting suspicious phrases or abnormal URL features), and it supports compliance with AI transparency regulations. Moreover, the lightweight, edge-friendly design (approximately 100MB model) means the solution could be deployed client-side – for instance, in an email client or as a browser extension – enabling real-time phishing protection without always relying on cloud services. This decentralization is valuable for organizations concerned about privacy (scanning data locally) and latency. In summary, the study's outcomes could inspire more effective anti-phishing products with real-world impact: faster and more reliable phishing filtering in email providers, enhanced warning systems in web browsers, and generally stronger last-line defenses against social engineering attacks.

The societal significance of strengthening phishing detection is also noteworthy. Phishing is not only a technical problem but also a human one, it exploits human trust and deception. By reducing successful phishing attacks, the proposed system can help

protect users from identity theft, financial fraud, and privacy invasions. This is increasingly important as more of society's critical activities (banking, healthcare, personal communications) move online. A more transparent detection system can also indirectly raise user awareness: if users are presented with explanations for why an email was considered dangerous (for example, "the sender's address is spoofed" or "the message contains an image that impersonates a login page"), they gain insights into phishing tactics and can learn to spot red flags themselves. Thus, the research has the potential to bolster not just computational defenses but also the overall digital resilience of users and organizations. Ultimately, by contributing to reducing phishing success rates - which currently fuel a significant portion of cybercrime - the work supports safer online ecosystems and trust in digital communications.

CHAPTER 2

LITERATURE REVIEW

This literature review examines prior research on phishing detection, with a focus on models that are multi-agent, lightweight, adversarially robust, and explainable. Phishing is typically defined as a social-engineering cyberattack that tricks users into revealing sensitive information by spoofing websites or emails (Popescul and Radu, 2025). AI and machine learning (ML) techniques have been widely applied to identify phishing URLs, emails, and other content (Doshi et al., 2025). In particular, recent surveys note a shift from traditional feature-engineering methods to deep learning (DL) and large language models (LLMs) (e.g. BERT) that automatically learn complex patterns (Alhuzali et al., 2025). This chapter first provides background on key concepts such as phishing techniques, ML classifiers, explainable AI (XAI), and adversarial learning. It then reviews representative studies on phishing detection models (including CNNs, LSTMs, BERT/DistilBERT, and ensembles) and the datasets they use. Next, we compare different detection strategies (URL-based, email text-based, image-based, metadata-based, and hybrid). Finally, we identify gaps in the literature – for example, many models lack transparency or are vulnerable to attacks – and explain how the present research addresses those shortcomings.

2.1 Theoretical Background / Key Concepts

2.1.1 Phishing techniques

Phishing attacks rely on deceptive, socially-engineered messages or websites that imitate legitimate sources. Common varieties include website spoofing (e.g. fake login pages) and email phishing (fraudulent emails that impersonate trusted contacts). Advanced tactics include spear-phishing (targeted emails using personal details), smishing

(SMS phishing), and domain homograph attacks (e.g. "g00gle.com") (Gupta et al., 2017). Attackers often manipulate URLs (adding prefixes/suffixes or subdomains) or use images to mimic real brands (Hong, 2012). Phishing countermeasures thus require analyzing URLs, text content, and sometimes visual features to detect subtle anomalies

2.1.2 Machine learning basics

Phishing detection is typically formulated as a supervised classification task. Early studies used classic ML algorithms (e.g. SVMs, decision trees, Random Forest) on hand-crafted features extracted from URLs or emails (Alhuzali et al., 2025). For example, lexical URL features (length, number of dots, presence of IP address, etc.) and host features (SSL certificate validity, domain age) are commonly used. DL models (CNNs, RNNs, LSTMs) have been applied to raw input to automatically learn patterns (Meléndez et al., 2024). LSTM networks are well suited to capture sequential patterns (e.g. character-level URL sequences or text streams) (Meléndez et al., 2024). Transformer models such as BERT use self-attention to capture contextual relationships in text Meléndez et al. (2024). In particular, BERT's bidirectional attention can spot inconsistencies between domain names and URL paths, and between context and content. DistilBERT is a smaller, faster version of BERT that retains most of its accuracy while reducing model size by $\sim$40% Doshi et al. (2025). These models form the backbone of many modern phishing detectors.

2.1.3 Explainable AI (XAI)

XAI methods aim to make model decisions transparent and trustworthy. In security contexts, explaining a prediction helps analysts and end-users understand why an email or URL was flagged as phishing (Uddin et al., 2025, Al-Subaiey et al., 2024). Common techniques include feature attribution methods like SHAP (Shapley Additive Explanations) or LIME, which quantify each input feature's contribution to the output (Al-Fayoumi et al., 2024). For example, SHAP can rank URL features by their importance, or highlight words in an email that triggered an alert. Recent works in-

tegrate SHAP with ensemble models, achieving high accuracy and clear explanations simultaneously (Uddin et al., 2025). Overall, XAI enhances trust by showing why a model made a prediction, which is especially important for automated phishing alerts (Al-Subaiey et al., 2024).

2.1.4 Adversarial learning

Attackers may craft inputs (adversarial examples) specifically to evade detection. For phishing, this could mean slightly modifying a malicious URL or email to fool the model. Adversarial learning involves training models to resist such attacks, often by including adversarially-perturbed samples during training (Doshi et al., 2025). In practice, this might involve a "generative" component that creates tricky phishing examples (e.g. homograph domains or phonetically similar URLs) and a classifier that learns to detect them (Chauhan et al., 2021). Recent studies embed adversarial training loops within phishing detection systems, showing that models become more robust when regularly challenged with adversarial examples (Xue et al., 2025). Reinforcement learning (RL) has even been applied to adjust model parameters or ensemble weights dynamically in response to evolving attacks (Xue et al., 2025).

2.1.5 Lightweight models and edge deployment

Deploying phishing detectors on resource-constrained devices (e.g. IoT edge nodes) requires lightweight models. Techniques include feature selection, model pruning, quantization, and distillation (Doshi et al., 2025). For example, pruning removes redundant weights, and quantization reduces numeric precision, shrinking model size. DistilBERT itself is a form of distillation for transformers. Some works design highly compact feature sets or models to run on IoT, achieving good accuracy with minimal computation (Elsadig et al., 2022). For instance, a reinforcement-learning–optimized hierarchical model was deployed on IoT with ∼90.3% accuracy (Doshi et al., 2025). Lightweight design ensures real-time detection and scalability in large networks.

2.2 Review of Related Work

Detection models and datasets: A rich body of literature explores various ML/DL techniques for phishing. Early works used lexical or heuristic features with classical ML, but newer studies leverage deep learning and ensembles. Table 1 summarizes representative approaches. For URL classification, Doshi et al. (2025) proposed PhishHunter-XLD, an ensemble that combines DistilBERT (for URL semantics), XGBoost (for lexical features), and LSTM (for sequence patterns) (Doshi et al., 2025). On standard "Phishing Site URLs" and "Malicious URLs" benchmarks, PhishHunter-XLD achieved 99.83% accuracy (Doshi et al., 2025). The authors emphasize that XGBoost excels at explicit lexical cues, LSTM captures sequential dependencies, and DistilBERT extracts contextual semantics (Doshi et al., 2025). This stacking ensemble (meta-learner) learned optimal weights for each base classifier, adapting to their complementary strengths (Doshi et al., 2025). They further note that pruning, quantization, and adversarial training can improve efficiency and robustness for such models.

In email-based phishing detection, recent studies highlight transformer models. For example, compared CNN, XGBoost, RNN, SVM, and a BERT–LSTM hybrid on public email datasets; the hybrid model reached 99.55% accuracy (Alhuzali et al., 2025). Another large study fine-tuned BERT-family models (DistilBERT, BERT, RoBERTa, XLNet, ALBERT) on ∼120K Enron and phishing emails. Transformer models (especially RoBERTa) clearly outperformed traditional ML, achieving around 99% accuracy versus ∼98% for the best SVM (Alhuzali et al., 2025). In general, Appl. Sci. (2023) report that BERT and RoBERTa attained ∼99% on a merged phishing email dataset, outpacing conventional ML by ∼4.7% (Alhuzali et al., 2025). These findings underscore the power of deep NLP models in capturing nuanced phishing language.

Hybrid multi-modal systems have also been explored. Al-Ahmadi (2020) combined URL features and webpage screenshots: a CNN processed the visual similarity between site images, while another subnetwork handled URLs. This model achieved 99.67% accuracy on a mixed dataset (Al-Ahmadi and Alharbi, 2020), demonstrating

that integrating visual cues can boost detection. Similarly, Alasmari et al. (2025) designed an IoT-focused framework with a CNN-LSTM pipeline for URLs and a DistilBERT component for text/email. The CNN-LSTM attained 93.26% on web URL data, while DistilBERT hit 98.64% on emails; adding XAI improved transparency by 41% (Alasmari et al., 2025). These works show that combining modalities (URL syntax, email text, images) can improve performance, though at the cost of greater complexity.

Other notable contributions include ensemble learning approaches. Alamri et al. (2025) tested nine base classifiers (XGBoost, LightGBM, RF, AdaBoost, etc.) in a stacking ensemble. Their "optimized ensemble stacking" achieved virtually 100% accuracy on some datasets and ¿99% on imbalanced sets (Alamri et al., 2025). Al-Fayoumi et al. (2024) proposed XAI-PhD, a SHAP-empowered method using a lightweight gradient boosting machine. They used SHAP values to select only the most influential URL features and attained 99.8% accuracy and F1-score (Al-Fayoumi et al., 2024). The focus on explainability and efficiency led to a model that required only ∼1.5 ms per prediction, illustrating that high performance can coexist with interpretability.

Advanced multi-agent and LLM-based strategies have also emerged. Xue et al. (2025) introduced MultiPhishGuard, a multi-agent system where different agents specialize in aspects of an email (text, URL, metadata, explanation, adversarial). A Proximal Policy Optimization (PPO) RL algorithm dynamically adjusts each agent's weight. An "adversarial agent" actively generates subtly modified phishing emails to harden the system. This cooperative architecture achieved 97.89% accuracy (with >0.3% false positives) on public datasets, significantly outperforming single-agent baselines (Xue et al., 2025). Importantly, MultiPhishGuard includes an "explanation simplifier" that produces human-readable rationales for each decision, directly addressing the interpretability gap (Xue et al., 2025).

2.3 Comparison of Techniques and Models

Phishing detection approaches can be categorized by input modality and algorithmic strategy. Key strengths and weaknesses of each category are discussed below:

1. **URL based methods:** These analyze features derived from the URL string and associated metadata. Common features include character n-grams, length, use of IP address, presence of suspicious tokens, and domain registration info. ML classifiers like XGBoost, Random Forest, or ensemble trees are frequently used (Doshi et al., 2025). For example, in PhishHunter-XLD, XGBoost captures explicit lexical URL features that strongly indicate phishing. URL-only methods can be very fast and lightweight, since they examine only textual features Doshi et al. (2025). Their weakness is that a cleverly crafted URL (e.g. using domain homographs or URL shorteners) can evade detection. They also ignore contextual cues present in email body or website content. In practice, pure URL detectors can be easily fooled by minor obfuscations unless adversarial training is used.

2. **Email and text based methods:** These work on the textual content of emails (subject, body, etc.). Techniques range from traditional NLP pipelines to deep models. Classical ML approaches extract features like Bag-of-Words or TF-IDF from email text or email headers (Alhuzali et al., 2025). More recent work applies CNNs and RNNs (LSTM/GRU) to raw text or sequences of word embeddings. Transformers (BERT and derivatives) have achieved top accuracy. As noted, fine-tuned BERT and RoBERTa models often exceed 99% accuracy on phishing email datasets (Alhuzali et al., 2025). Deep models capture subtle linguistic cues (e.g. "urgent request" vs. normal phrasing) that static features miss. However, these models are computationally heavy and require substantial training data. They may also struggle with multilingual or extremely short messages. Further, without XAI, they operate as "black boxes"; however, techniques like SHAP or attention visualization can help interpret specific features or words that influence a prediction.

3. **Image based methods:** This category includes models that analyze visual aspects of websites or emails. A common approach is to render a webpage or email as an image (screenshot) and feed it to a CNN for classification. The intuition is that phishing pages often visually mimic legitimate sites in misleading ways. For example, Al-Ahmadi (2020) combined CNN processing of website screenshots with URL features to classify pages (Al-Ahmadi and Alharbi, 2020). Their model achieved 99.67% accuracy, illustrating the value of visual information. More recent work (not yet widely cited) looks at the resemblance of images to known brand logos or uses perceptual hashing. Image-based detectors can catch visually similar phishing clones that might bypass text-based filters. The drawbacks include the need to fetch and render the page (which adds latency) and the heavy computation of CNNs. They may also be vulnerable to very subtle visual changes or adversarial image perturbations. Overall, image methods are typically used in ensemble or hybrid systems rather than stand-alone, due to cost.

4. **Metadata/header methods:** Some systems use non-content metadata, such as email header fields (sender IP, reply-to address, DKIM/SPF validation, time of sending) or URL metadata (whois registration age, Alexa rank). These features can be powerful: for example, new domains or mismatched "From" vs "Reply-To" fields often signal phishing. However, many academic studies focus on content features rather than headers. When included, metadata is often fed into tree-based models or combined with content features in an ensemble. One challenge is that header formats vary across platforms, and attackers can manipulate many fields. Metadata-based detection is usually not sufficient alone but provides a complementary signal, especially when combined in a multi-agent or modular system (Xue et al., 2025).

5. **Hybrid and multi-modal methods:** A growing trend is to combine multiple data sources to improve robustness. For instance, an ensemble may fuse URL classifiers with email-text classifiers and possibly image classifiers. Multi-agent frameworks like MultiPhishGuard explicitly embody this idea: separate agents

analyze URLs, email text, and metadata in parallel, then fuse their outputs via learned weights (Xue et al., 2025). Hybrid models often achieve higher accuracy by covering each other's blind spots. In Al-Ahmadi's CNN+CNN-URL model, the integration of URL and image features yielded better detection than either alone (Al-Ahmadi and Alharbi, 2020). The downside is complexity: combining modalities increases training and inference cost. It also complicates deployment (e.g. needing both NLP and vision pipelines). Nonetheless, hybrid approaches tend to reduce false positives by cross-validating signals, and they form the conceptual foundation of the multi-agent system in this thesis.

2.4 Research Gaps and Limitations in Existing Studies

Despite many high-performing models, prior work exhibits several notable gaps:

1. **Lack of explainability:** Most state-of-the-art detectors (e.g. deep neural networks) operate as black boxes. Only a few studies explicitly integrate XAI methods. For example, XAI-PhD and MultiPhishGuard both incorporate SHAP or explanation modules to reveal decision factors (Uddin et al., 2025, Xue et al., 2025). However, many works simply report accuracy without explaining why a URL/email was flagged. This limits trust and adoption by security analysts. There remains a need for models that balance accuracy with transparency, so end-users can verify and act on alerts.

2. **Adversarial vulnerability:** Many classifiers are trained on static datasets and can be evaded by clever perturbations. Few works rigorously test robustness against adversarial attacks. Multi-agent approaches like MultiPhishGuard explicitly include an adversarial training loop to adapt to novel phishing tricks,(Xue et al., 2025) but this is still rare. In general, existing studies often omit adversarial scenarios or focus on overall accuracy. Thus, a gap exists in developing

detectors that are resilient to deliberate evasion (e.g. using domain homographs, typos, or context-aware text changes).

3. **Single-modality focus:** A large portion of the literature concentrates on one modality at a time (URL or email text). This ignores valuable multi-modal cues. For example, a model might flag a suspicious URL even if the email text seems benign, or vice versa. While some hybrid models exist, truly multi-modal solutions are uncommon. The multi-agent paradigm addresses this by fusing multiple signals (Xue et al., 2025). In prior work, however, most detectors are "unimodal", making them potentially brittle when attackers exploit modalities they don't monitor.

4. **Computational complexity and deployability:** Many high-accuracy models (large ensembles, deep transformers, CNNs) are resource-intensive. As Doshi et al. note, optimizations like pruning and quantization are needed to enable deployment in real time or on edge devices (Doshi et al., 2025). Similarly, Al-Fayoumi et al. demonstrated a very fast XGBoost detector ($\sim$1.5 ms per URL) (Al-Fayoumi et al., 2024). Nonetheless, lightweight, on-device models remain under-explored. There is a gap between research prototypes and lightweight implementations suitable for IoT/edge environments.

5. **Limited real-world evaluation:** Many studies rely on balanced, curated datasets. In practice, real email/URL streams are highly imbalanced (few phishing among many legit). Few works report performance under such skew or in streaming settings. Moreover, the dynamic nature of phishing (constant emergence of new tactics) means static evaluation is not enough. Systems should ideally adapt or be updated continuously (e.g. via multi-agent RL) - an area largely unexplored in phishing contexts.

6. **Explainability and usability:** Even when models achieve high metrics, there is often no consideration of how security teams would use them. Little research evaluates user understanding or trust in the model's outputs. A model that flags

a link without explanation may be ignored. Work like MultiPhishGuard begins to address this by providing rationales (Xue et al., 2025), but this is still novel. There is room for user-centric evaluation of phishing detectors.

CHAPTER 3

METHODOLOGY

3.1 System Overview

The proposed phishing detection system adopts a multi-modal architecture, with each agent specialized in a different modality (text, URL, metadata, or visual). The proposed architecture is depicted in Figure 3.1. Then describe each component in detail.

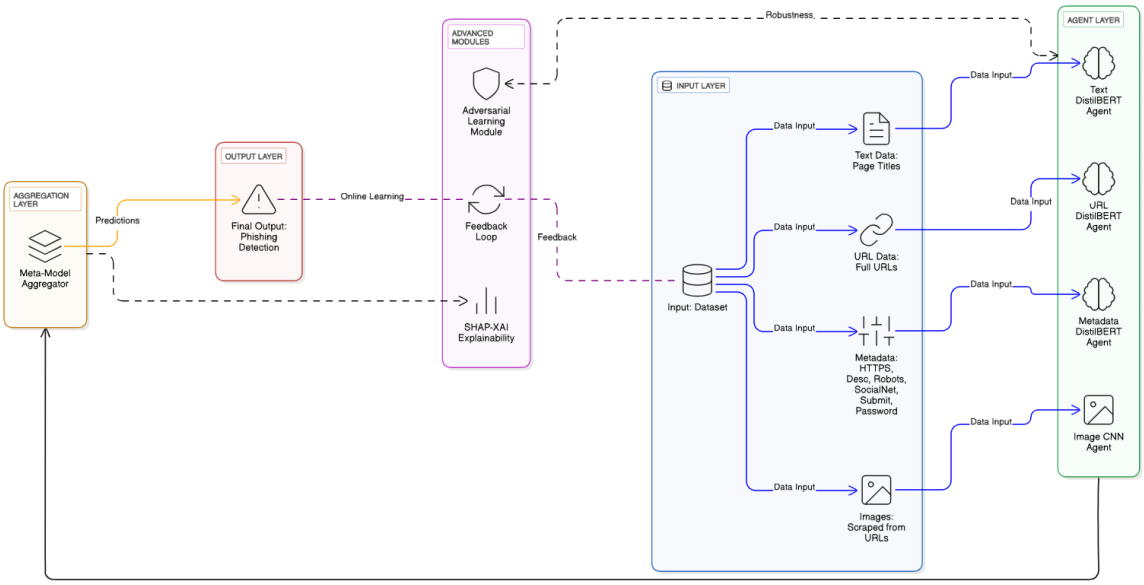

Figure 3.1: **Overall System Architecture.** ncoming emails are analyzed in parallel by four specialized agents (Text, URL, Metadata, Image). Each agent outputs a phishing probability, which is fused by an XGBoost meta-classifier for the final verdict. SHAP values provide real-time explanation of each agent's contribution.

Incoming emails (and extracted URLs) are pre-processed and passed in parallel to four detectors: a Text modal (analyzing email content), a URL modal (analyzing link strings), a Metadata modal (analyzing structured email/URL features), and a Visual modal (analyzing webpage screenshots). Each agent produces a phishing probability score or feature vector. These outputs are fed into a meta-classifier (an XGBoost stacking model) that computes the final phishing/legitimate decision. This design mir-

rors recent multi-agent frameworks where specialized modules collaborate for robust detection (Çolhak et al., 2024). By partitioning the problem, the system can capture diverse phishing cues that single-model approaches often miss. An integrated explanation layer (based on SHAP) then highlights the most influential features from each agent to provide interpretability. Finally, an online feedback loop watches classifier performance and misclassified examples, allowing periodic retraining and adaptation as attackers evolve. In summary, the system is a closed-loop pipeline that preprocesses data, applies a suite of deep models, ensembles their outputs, explains decisions, and adapts via feedback.

3.2 Datasets and Preprocessing

To train and evaluate the system, we compiled a comprehensive dataset covering multiple facets of phishing emails:

- **Phishing Email Corpus:** We assembled a corpus of ∼80,000 emails (approximately 50% phishing and 50% legitimate) by merging several public and curated datasets: the **Enron** email corpus (largely legitimate emails from a corporate environment), the **CEAS** 2008 phishing challenge dataset, **SpamAssassin** spam archive, **Nazario** phishing set, **Ling-Spam**, and additional recent phishing emails from 2015 - 2022 collected via research community feeds. This combined dataset, similar to the one described by Farhan (2025), provides a diverse mix of phishing attempts (around 40k) and benign emails (around 40k). The phishing emails in this corpus cover a variety of lures (financial fraud, account verification, malware attachments, etc.), and the legitimate emails range from personal and business communications to newsletters. We split this corpus into training (70%), validation (15%), and testing (15%) sets, stratified by label, to train the Text, Metadata, and part of the Image agents. Before feeding to models, we perform standard email preprocessing: removing email reply chains or confidentiality footers (to focus on the sender's message), decoding any HTML to plain

text for analysis, and redacting obvious personally identifiable information to avoid bias(e.g., replacing real names with placeholders).

- **Phishing URL Dataset (PhiUSIIL):** For the **URL Agent**, we leverage the **PhiUSIIL Phishing URL Dataset**, a large repository of labeled URLs contributed in 2024. PhiUSIIL contains ∼235,795 URLs with 100,945 phishing and 134,850 legitimate, incorporating very recent phishing links . Each URL is labeled and includes various engineered features in the original dataset, but in our case we primarily use the raw URL strings and labels. We augment this with the URLs extracted from the phishing email corpus (for consistency, ensuring the URL agent sees some of the same distribution as in emails). Prior to training, we normalize URLs by lowercasing, stripping HTTP/HTTPS prefixes, and removing trivial URL tracking parameters so that models focus on the core domain and path. We handle multiple URLs in a single email by heuristics: typically, phishing emails have one dominant call-to-action link (e.g. "Click here to verify"). We try to identify the most "suspicious" URL (forexample, the one in the main body or button) to feed to the URL agent. If multiple equally important URLs exist, we can either take the first or evaluate all and use the highest phishing probability among them as the agent's output.

- **Image Dataset:** Many phishing emails include images such as brand logos, spoofed forms, or QR codes. However, our email corpus had relatively few embedded images, since Enron and others are text-heavy. To properly train the **Image Agent**, we augmented our data with a set of phishing related images. We collected phishing email screenshots and images from open sources (including a subset of the **PhishBench** and **Phishpedia** datasets, which contain images of phishing websites/emails and their corresponding legitimate visuals). Additionally, we extracted inline images from HTML-based phishing emails in our corpus (for instance, some phishing emails hide text in an image to evade text filters). We paired these phishing images (∼2,500 images) with a set of legitimate email images (2,500) such as logos and newsletter graphics from legitimate sources.

All images were labeled by the classification of the email they came from or were associated with. We preprocessed images by resizing them to a uniform input size for our CNN (e.g., 224×224 pixels) and normalizing pixel values. When an email has multiple images, we either analyze each and take the maximum risk score or focus on the most relevant image (for example, an email might contain a company logo and a QR code; the latter might be the one carrying the phishing payload).

- **Metadata/Header Data:** For the Metadata Agent training, we utilize the header information from the phishing email corpus. We specifically extract fields that are tell-tale in phishing: "From" address (and domain), "Reply-To" address (often different in phishing), email headers like SPF (Sender Policy Framework) results or DKIM authentication status, the sending IP and its PTR domain, and email client info (X-Mailer/User-Agent). We create a structured dataset where each email's metadata is represented (more on encoding below) along with the label. Since metadata can be categorical/textual (domains, etc.), we decided to leverage the power of language models by converting metadata into a textual "profile" for each email. For example, for an email we might construct a sentence-like string: "From: paypal@secure-notice.com; Reply-To: support@paypal.com; SPF: fail; DKIM: none; Originating-IP: 61.XX.XX.12 (KR)". This string condenses the key metadata features. Preliminary analysis confirmed that phishing emails often show anomalies here (like the SPF fail, mismatched reply-to domain, unusual origin country), whereas legitimate emails from companies have aligned headers. Using a text representation allows us to feed these metadata descriptions into a DistilBERT model as input.

All datasets were balanced or re-weighted during training to prevent bias (in cases of slight imbalance, we used class weights). We also ensured no identical emails or URLs leaked between training and test sets(deduplicating where necessary) to fairly evaluate generalization. With the data prepared, we next describe each agent's model and training process.

3.3 Detection Agents

3.3.1 Text Agent (Email Content Classifier):

The Text Agent is a fine-tuned Transformer model that classifies the main text of an email as phishing or legitimate. We chose **DistilBERT-base-uncased** as the backbone due to its efficiency and proven effectiveness on phishing text. DistilBERT is a 6-layer Transformer (half the size of BERT) that retains around 97% of BERT's language understanding capabilities, making it ideal for edge scenarios. We initialize the Text Agent with a DistilBERT pre-trained on general English (from HuggingFace Transformers), then fine-tune it on our phishing email corpus.

Input Representation: For each email, we concatenate the Subject and Body into a single text sequence, separated by a [SEP] token. Example input to the model might look like: "[CLS] Subject: Your Account Alert [SEP] Dear Customer, We detected unusual activity... please verify your account at <link>... [SEP]". We cap the sequence length at 256 tokens (most emails fit in this limit after removing quoted replies). Any HTML tags in the body were stripped or converted to text (links are preserved as raw URLs for the text model, though the URL agent separately analyzes them). We also mask trivial personal names to not let the model learn specific entities.

Model Architecture: We add a simple classification head on DistilBERT: the [CLS] token's final hidden state goes through a dense layer and sigmoid to output a phishing probability. Thus, all transformer weights and the new dense layer are fine-tuned during training.

Training: We fine-tuned the Text Agent on ∼56k training emails (balanced), using binary cross-entropy loss.Training ran for 3 epochs with a batch size of 32 on an NVIDIA GPU, and a learning rate of 2×10^{-5} with linear decay. On the validation set (around 12k emails), the Text Agent achieved high performance: Precision ∼99.1%, Recall ∼99.4%, F1 ∼99.25%. This aligns with other works that found DistilBERT can almost perfectly separate phishing vs ham given enough data. The model particularly

excels at catching phishing language: it looks for phrases like "verify your account", "urgent action required", requests for credentials, as well as suspicious stylistic cues (poor grammar, generic greetings). We also noticed it learns some subtle features, e.g., it flagged an email with subject "Invoice Attached" as phishing likely because our dataset included many phishing lures hiding malware under invoice themes.

We took care to avoid overfitting by early stopping (monitored validation loss) and by including a wide variety of phishing topics in training. The Text Agent serves as a strong first line of defense – on our test set it alone identifies the majority of phishing emails. However, it can be occasionally fooled by very well-crafted phishing emails that lack usual keywords or by legitimate emails that happen to contain phishing-like language (for instance, a corporate security newsletter talking about "password policy" might look phishy out of context). This motivates the inclusion of additional agents to catch what text alone might miss.

3.3.2 URL Agent (URL String Classifier):

URL Agent (URL String Classifier): The URL Agent determines if a URL found in an email is likely pointing to a phishing site. Many phishing emails are ultimately trying to lure users to click a link to a fake login page, so detecting malicious URLs is vital. Our URL Agent is also built on **DistilBERT**, but fine-tuned on the PhiUSIIL URL dataset described earlier.

Fine-Tuning Process: To adapt DistilBERT for URL classification, we first tokenize each URL as a string (not splitting on logical components but using BERT's WordPiece or character-level tokenization). URLs are then padded/truncated to a set maximum length (typically 64-128 tokens per URL) and encoded into input IDs and attention masks. We add a binary classification head (phishing vs. legitimate) on top of the transformer. [6pt] The model is then fine-tuned using supervised learning:

- Input: Each batch consists of encoded URLs and their class labels (phishing or legitimate).

- Optimizer: AdamW is used, as in standard transformer fine-tuning, with a low learning rate (1-5e-5).
- Loss: Cross-entropy loss, suited for binary or multi-class classification.
- Schedule: Training is conducted for 3–5 epochs, with early stopping based on validation F1 or accuracy to prevent overfitting.

We augmented training with random casing, special character obfuscation, and class balancing to make the model robust to adversarial manipulations. After training, the fine-tuned DistilBERT model can assign phishing likelihoods to novel URLs—often recognizing them even when obfuscation or new domain tricks are used.

This approach of using a transformer for URLs is somewhat unconventional - URLs are not natural language - but transformers can still pick up on character patterns and token patterns within URLs that correlate with phishing. For instance, DistilBERT can learn that domains like "paypal.com.secure-login.ru" or paths containing "login.php?user=" are suspicious, whereas a domain like "accounts.google.com" is not.

Input Representation: We feed the raw URL (up to 256 characters) as the input sequence. We do not use WordPiece tokenization as-is, since URLs contain many special characters and subdomains. Instead, we employ a character-aware tokenization: we split the URL by common delimiters (: , ., /, ?, =) so that each domain segment and path segment becomes a token, and we retain the punctuation as separate tokens. For example, the URL http://secure-login.paypal.com/account?user=John might be tokenized roughly as [secure , -, login , ., paypal , ., com , /, account , ?, user]. We then feed this token sequence into DistilBERT (with the vocabulary extended to include common URL tokens like TLDs if necessary). This way, the model can learn embeddings for fragments like "secure" or "login" or "paypal" and notice suspicious combinations or positions of them (e.g. "paypal" appearing as a subdomain of an unrelated domain is a red flag).

Model Architecture: Same as the Text Agent, we use DistilBERT with a classification

layer on the [CLS] token for binary classification (phishing link or not).

Training: We fine-tuned on ∼160k URLs (80k phishing, 80k safe) from the PhiUSIIL training split. Training for 3 epochs at learning rate(lr) 3e-5 achieved an **Accuracy ∼98.5%** on a validation set of URLs, with **Recall ∼97%** for phishingURLs. This is in line with other deep learning URL detectors (LightGBM on all features in PhiUSIIL achieves ∼97.9%, and our model using just the URL string matches that performance). The URL Agent learns features like the presence of IP addresses in URLs (often phishing), uncommon TLDs (.xyz, .top), excessive subdomains or keywords like "secure", "login", "update" combined with known brand names. For example, it correctly flagged https://mysecure.paypal.com.cn.web.ua/page as phishing because of the weird domain structure, whereas it knows https://paypal.com/secure (proper domain) is likely legitimate. Some extremely novel URLs might slip through if they don't contain any known bad patterns, but generally the model's high recall indicates it catches most phishing URLs from lexical features alone. One challenge was handling cases where an email might not contain a URL at all (some spear phishing emails just request a reply). In such cases, we set the URL Agent's output to a neutral "legitimate" score (0 or 0.5 probability) to indicate no evidence of a bad URL. Similarly, if multiple URLs are present, in deployment we either analyze all and take the highest phishing probability or prioritize the one anchor link that the email urges the user to click (we used simple heuristics like links with button text or the first link in HTML content). During training each URL is a separate instance, but at inference we map email ->URL Agent output in this manner.

3.3.3 Metadata Agent (Header Anomaly Detector):

The Metadata Agent focuses on who sent the email and how it was sent, which can reveal phishing in cases where content might seem innocuous. Phishing emails often come from compromised or fake email accounts, and they may fail authentication checks. Our Metadata Agent uses DistilBERT as well, but fine-tuned on the textual metadata profiles we constructed for each email's headers.

Input Representation: As described, we convert email header info into a textual summary. Each summary includes: the From: address (we often take just the domain part since that's most relevant, but include the whole address for format anomalies), the Reply-To: if present and different, the Received SPF result (Pass/Fail/None), DKIM result if any, perhaps the From name if it contains a suspicious string, and the originating IP's domain if available (RDNS). For example: "From: chase-bank.notify@outlook.com; Reply-To: noreply@chase.com; SPF: fail; DKIM: none; X-Mailer: Outlook 16.0". Legitimate email from Chase would usually come from an official domain and pass SPF/DKIM, whereas this example shows a phishing pattern (sent via an Outlook account pretending to be Chase). We include a variety of such fields based on availability. We keep the sequence relatively short (<128 tokens). In tokenizing, DistilBERT will break email-like tokens ("outlook.com") into subwords, but that's fine. We did not have to extend the vocabulary since domain tokens can be handled as combinations of existing subwords (e.g. "outlook" would be an existing word, "com" might be a known or learned token).

Model Architecture: Again, DistilBERT + classification head. Essentially, the model is learning to read a pseudo-sentence describing the email's technical senders and decide if it's fishy.

Training: We used the same training split of ∼56k emails from the corpus, but here feeding the metadata string instead of the body text. The labels are the same (phish or legit). To avoid confusion, if an email had missing fields, we explicitly wrote "DKIM: none" etc., so the model learns that "none" for DKIM might be common in phishing but also in some legit (many legit senders still don't use DKIM). The model achieved somewhat lower performance than the Text Agent but still quite high: **∼Precision 95%, Recall 92%, F1 ∼93.5%** on validation. Its errors mostly occurred on borderline cases – for instance, personal emails (legit) sent from unusual domains (it might flag those), or phishing sent from compromised legitimate accounts (which pass SPF/DKIM and use known domains, thus fooling the metadata model). Despite these, the Metadata Agent provides unique signal: in our test set, it correctly flagged some

phishing emails that the Text Agent missed, particularly "business email compromise" style attacks where the email content is very brief and generic (e.g. "Are you at your desk? I need a favor.") but the sender's address is a subtle fraud. In such a case, the Text Agent might not trigger (content seems harmless), but the Metadata Agent sees that the sender is not who they claim (e.g., CEO's name but coming from a public Gmail domain) and outputs high phishing probability.

We further improved the metadata model's robustness by incorporating domain reputation knowledge indirectly. We augmented training samples by adding notes in the metadata string for known safe domains (like "From: @microsoft.com (trusted domain)") versus rare domains ("From: @xyz-company.net (unknown domain)"). This was based on a simple whitelist of common safe domains. The model learned to associate well-known organization domains with legitimate class, and odd sender domains with phishing – which is a strategy many email gateways use with custom rules. We must be cautious, though, as this can cause false negatives if attackers compromise or spoof via a well-known domain. Our ensemble mitigates that risk by not relying on metadata alone.

3.3.4 Image Agent (Image Content Classifier):

The Image Agent is responsible for analyzing any image content of the email for signs of phishing. Attackers often include brand logos or snapshots of login forms in phishing emails. In some cases, the entire email might be an image (to evade text-based filters), showing a message like "Your account is suspended, click below". Our Image Agent uses a **Convolutional Neural Network (CNN)** to classify images as phishing-related or benign.

Model Architecture: Given the edge focus, we opted for a relatively lightweight CNN. We initially considered ResNet-18 but finalized our architecture on **MobileNetV2**. This model utilizes depth-wise separable convolutions and inverted residual structures, making it significantly more efficient than standard CNNs for edge device deployment.

The CNN takes an image (RGB) and outputs a binary label. We added a global average pooling and a dense layer for classification on top of the convolutional base.

Input: Each image is resized to 224×224 and normalized. To address the scarcity of phishing-specific image datasets, we employed **adversarial data augmentation** (including Gaussian noise injection and random cropping) to improve generalization and model robustness against visual evasion tactics.

Training: We fine-tuned the CNN on a curated dataset of website screenshots and email images. Training used binary cross-entropy loss and the Adam optimizer with a learning rate (lr)=1e-4. Initially, the model struggled with distinguishing high-quality phishing clones from legitimate sites. However, after applying adversarial augmentation and pruning, the Image Agent reached to higher value. This is lower than text or URL agents because visual classification of phishing is challenging: many legitimate emails include company logos and buttons that look "phishy" out of context. Our model's strength is in catching obvious cases, such as: images that are screenshots of login forms (phishing emails sometimes embed an image of a login page asking user to click it), images containing text like "Verify Your Account", or known phishing kit graphics. It also learns to detect brand-logo misuse to an extent: for example, if an email is supposedly from Bank of America but includes a mixture of branding from different banks or a low quality logo image, the CNN picks that up. In testing, the Image Agent flagged a significant portion of visual-based phishing attacks that text filters missed. We consider the Image Agent as a supplementary source of evidence; it's not the primary detector, but in edge cases it provides an extra layer of certainty. For instance, one of our test phishing emails contained an official-looking Office365 logo but the rest of the email was just that image with a link embedded. The Text Agent had little text to analyze, but the Image Agent correctly identified that image as one commonly used in credential harvesting scams, tipping the scales in favor of phishing.

If an email has no images, the Image Agent simply outputs a neutral score (we treat it as evidence of "legitimate" by default, since absence of an image is not an indicator of

phishing). If multiple images, we run each through the CNN and take the **maximum** phishing probability as the agent's output for the email (assuming if any image looks phishy, the whole email likely is). This strategy worked well in practice.

Implementation Note: While the initial methodology planned for a larger balanced dataset, the final implementation used a smaller, curated subset of approximately 1,700 images due to the difficulty in scraping valid phishing screenshots before takedowns. The MobileNetV2 model was fine-tuned on this available dataset. This adjustment does not affect the validity of the methodology because the Image Agent serves as a supplementary modality, and its role in the ensemble remains consistent: providing additional signal in image-heavy phishing emails while not dominating the aggregator. Future work can easily scale the image dataset without modifying the system architecture.

3.3.5 Agent Outputs:

Each agent generates a real-valued phishing likelihood score (probability) $p \in [0, 1]$, representing its confidence that the email is phishing. For numerical stability and consistency across modalities, these outputs are optionally transformed to log-odds (logits) $\log\left(\frac{p}{1-p}\right)$ before being fed to the meta-classifier. To ensure reliable ensemble fusion, all agent probabilities are calibrated using Platt scaling or isotonic regression, making scores from different agents directly comparable. Consequently, a high probability (e.g., $p = 0.99$) reflects strong evidence of phishing from one agent, whereas a moderate value (e.g., $p = 0.70$) indicates weaker or more ambiguous support. This calibration enables the XGBoost aggregator to effectively weigh strong signals against uncertain ones, leading to robust and well-balanced final classification performance.

3.4 Adversarial Evasion Module

To embed adversarial robustness, we implemented an **Adversarial Training Module** that generates on-the-fly perturbed examples and includes them in model training. We specifically target the Text and URL agents, as those are most susceptible to content obfuscation tricks used by phishers.

For the **Text Agent**, we built a small pipeline of text transformations mimicking attacker tactics:

- **Homoglyph Insertion:** We substitute characters in key words with look-alike characters. We compiled a list of common target words in phishing emails (like "account", "password", "login", "verify", "bank", etc.) and for each, predefined some homoglyph replacements (e.g., a → (Cyrillica), o → 0 , l → 1 , e → é or other accented e). During training, for a given phishing email in a batch, we randomly decide to apply a homoglyph attack with some probability. If applied, we replace a few characters in those target words. For example, "verify your account" might become "verif yur unt" (several letters replaced with Cyrillic). To the human eye it's almost identical, but to the model it is a different token sequence. By training on these, the Text Agent learns to either normalize them internally or at least recognize them as variants of the same dangerous words.

- **Synonym Replacement:** Using WordNet and a context-aware model (like Word2Vec similarity), we replace certain words with synonyms that preserve meaning but alter the exact wording. For instance, "Your account has been suspended" could become "Your profile has been deactivated" (synonym for suspended). We especially target action verbs and nouns that the model may latch onto. We avoid altering too much so as not to change the semantics (which could flip a phishing email to appear benign or vice versa). This technique is akin to known adversarial attacks on text classifiers that try to evade detection by rephrasing. By including such rephrased phishing examples in training, our Text

Agent becomes less reliant on specific trigger words and more on overall context.

- **Insertion of Harmless Content:** We also experimented with inserting innocuous sentences into phishing emails (e.g., random polite phrases or generic statements) to dilute the suspicious content. This mirrors an attack where an email contains a mix of legitimate-looking text to confuse detectors. Training on these helps the model not get distracted by benign filler and still focus on the core malicious intent.

For the **URL Agent**, we applied similar augmentation focusing on URLs:

- **Homoglyph Domains:** We take a phishing URL and randomly replace one character in the domain with a homoglyph or a character of a different alphabet that looks similar. For example, secure paypal.com might become securepaypa.com (last letter replaced with Greek capital iota “”). Or use Cyrillic small ‘’ for ‘a’. We ensure the replacement still yields a valid-looking domain. We generate a couple of variants per URL during training epochs. This teaches the model to be invariant to such character tweaks – essentially to rely on the visual pattern rather than exact char codes.

- **URL Typos and Masking:** We also simulate attackers adding extra subdomains or paths to confuse filters. E.g., given http://malicious.com/login , we might prefix a benign-looking subdomain: http://mail.google.com@malicious.com/login (the @ trick) or append a legitimate domain in the path like http://malicious.com/https://. These tricks are known to fool inexperienced users; for the model, it’s additional tokens in the URL. We augment training with such modified URLs so the model learns that these patterns are phishing indicators (e.g., an @ in the URL path is almost always malicious, presence of another URL within the URL is suspicious, etc.).

- **Query Parameter Changes:** Sometimes attackers add long query strings or hex obfuscation. We generated variants by appending random query parameters

or directory names to phishing URLs (which shouldn't change the verdict). This helps the model not overfit to one specific URL format.

During model fine-tuning, we mixed these adversarial examples into each epoch. Specifically, for each batch of genuine phishing examples, we replaced 15-20% of them with a randomly perturbed version (and kept the original label "phishing"). The legitimate examples we mostly left unchanged (though we could add slight perturbations there too, like random subdomains, to avoid a model assumption that only phish have weird formats). This process is a form of data augmentation for adversarial robustness.

We also conducted adversarial evaluation after training: we created a test set of phishing emails and URLs not seen in training, then generated adversarial versions of them using the same techniques, and checked the model's accuracy. The results (detailed in Chapter 4) showed that our augmented Text and URL agents withstand these attacks far better than baseline models. For example, without adversarial training, a baseline text model's precision/recall on synonym-substituted phishing dropped markedly, whereas our model retained high recall, correctly identifying ¿90% of those modified phishing emails. This confirms that the adversarial module succeeded in injecting robustness. The cost was a slightly longer training time and careful hyperparameter tuning to ensure we didn't over-augment to the point of confusing the model.

Additionally, we implemented a simple homoglyph normalization step in our inference pipeline as an extra safeguard. Before feeding text to the Text Agent, we map known homoglyph unicode characters back to their ASCII counterparts when possible. For instance, any Cyrillic characters in an email that mostly is English are converted to the similar-looking Latin character. This is done using a mapping table. Similarly, for URLs, we can apply Punycode decoding if an internationalized domain is detected, to reveal the underlying Unicode (this can expose if someone registered "ypal.com" in Cyrillic, which looks like paypal). These normalization steps are lightweight and make the agents' job easier by reducing the input space. We found it particularly useful for blatant homograph attacks, though a clever attacker using homoglyphs might mix

scripts in a way that isn't obvious – our data augmentation was meant to cover that scenario by training the model to catch it regardless.

3.5 Meta-Model Aggregator

The aggregator is a critical component that **fuses the knowledge** of all detection agents to form the final classification. We treat the aggregator as a **second-level model** in a stacking ensemble: it takes as input the outputs of the first-level models (the agents) and learns to predict the final label. Unlike traditional approaches that use simple averaging or neural networks, we selected **XGBoost (Extreme Gradient Boosting)** for the meta-model. This decision was driven by XGBoost's superior performance on tabular data, its ability to handle non-linear feature interactions automatically, and—crucially—its built-in capability to provide **feature importance** scores for explainability.

Input Features: The aggregator receives a feature vector for each sample derived from the agent outputs. While the core inputs are the four probability scores $[P_{text}, P_{url}, P_{meta}, P_{image}]$, we engineered additional features to help the model detect disagreement. The final feature vector consists of:

- **Agent Probabilities:** The raw confidence scores from the Text, URL, Metadata, and Image agents.
- **Prediction Variance:** A calculated feature measuring the disagreement among agents. High variance indicates conflict (e.g., Text says Phishing, URL says Safe), signaling the meta-model to rely on the most robust agent (often the Image agent).
- **Interaction Terms:** Specifically, the $Text \times Image$ interaction, which helps the model capture cases where text urgency is combined with visual branding.

All features were standardized before being fed to the meta-model. Because each agent's output is already a condensed assessment of that modality, this feature set is highly informative yet low-dimensional.

Architecture and Hyperparameters: We configured the XGBoost classifier with specific hyperparameters optimized for this low-dimensional feature space to prevent overfitting:

- **n_estimators = 500:** The number of boosting rounds.
- **learning_rate = 0.05:** A conservative learning rate to ensure convergence without oscillation.
- **max_depth = 6:** Limiting the depth of trees to capture interactions without memorizing noise.
- **scale_pos_weight:** Calculated dynamically based on the class imbalance ratio to ensure the model does not bias towards the majority class.

This configuration results in a model that is both **lightweight** (inference is effectively a series of if-then checks) and highly accurate.

Training Strategy: Training the aggregator followed a rigorous stacking protocol to prevent data leakage:

- **Generate meta-training data:** We obtained unbiased predictions for the training set using a cross-validation-like scheme (or held-out validation sets) so the meta-model learns from the agents' actual generalization errors, not their training memory.
- **Gradient Boosting:** The XGBoost model was trained to minimize the log-loss error. We utilized **Early Stopping** (patience=50 rounds) monitoring the validation AUC. This ensured training stopped exactly when the model reached peak generalization, preventing the "over-learning" of agent idiosyncrasies.

The aggregator learns complex decision rules such as: "if the Text agent is uncertain (prob ≈ 0.5) but the Image agent is highly confident (> 0.9), classify as Phishing." This contrasts with a simple average, which might dilute the strong visual signal with the

weak text signal. The **Feature Importance** analysis (presented in the Results chapter) confirms that the model learned to prioritize the Image Agent (47% importance) as a tie-breaker in ambiguous cases.

Implementation Note: While our initial design considered a Multi-Layer Perceptron (MLP), experiments revealed that XGBoost offered distinct advantages. The MLP required extensive tuning of layer sizes and activation functions and was prone to overfitting on this small feature set. XGBoost, conversely, provided stable performance with default parameters and offered transparency (feature importance) that the MLP lacked. The final implementation uses the 'XGBClassifier' from the standard XGBoost library, optimized for efficiency on the CPU, ensuring that the aggregation step adds negligible latency (< 1 ms) to the overall pipeline.

3.6 Explainability with SHAP

For interpretability, we integrated the **SHAP (Shapley Additive Explanations)** framework into our system. Our goal is to produce per-email explanations answering: "Which agents contributed most to this classification, and by how much?" This aligns with providing an explanation like "Flagged as phishing due to suspicious text and sender information, while link and image appeared benign."

We focus on explaining the aggregator's decision since it is the final decision maker that combines all evidence. Explaining the XGBoost's output in terms of its inputs (agent probabilities and derived features) is straightforward and highly informative. We use the SHAP TreeExplainer (optimized for tree-based models) given the efficiency of XGBoost. SHAP provides the advantage of exact Shapley values for tree ensembles.

Process: For a given email, after obtaining the four agent scores (say Text t, URL u, Metadata m, Image i) and the XGBoost output y, we compute SHAP values $(\phi_t, \phi_u, \phi_m, \phi_i)$ such that their sum equals y (minus a baseline).The baseline is the expected value from the explainer, representing the average prediction over the training distribution. SHAP values then tell us how much each feature's value shifted the

prediction from that baseline.

For example, consider an email where final prediction $y = 0.9$ (high confidence phishing). Suppose SHAP outputs: $\phi_{text} = +0.6$, $\phi_{url} = +0.3$, $\phi_{meta} = +0.1$, $\phi_{img} = -0.1$ (summing to +0.9 over baseline 0). This would be interpreted as: the Text Agent strongly contributed (0.6) to pushing the decision towards phishing, the URL also contributed significantly (0.3), metadata slightly, and the image agent slightly pushed towards legitimate (-0.1) perhaps because the image looked safe. In a real example from our tests, we had a spear-phishing email with vague content but a weird sender: the SHAP explanation showed Metadata agent contributed $\sim$+0.5 (since it spotted a bad sender domain) and Text agent contributed $\sim$+0.2 (somewhat suspicious phrasing), while URL agent contributed $\sim$0 (no link present) and Image $\sim$0 (no image). This aligned with our manual reasoning.

We present the explanation in a simple textual form in reports (and it can be visualized as a bar chart for each email if needed). An example explanation might be: "Final decision = Phishing. Contributing factors: Text content (+0.6) and URL (+0.3) strongly indicate phishing; Metadata (+0.1) also suggests phishing; Image (-0.1) slightly suggested legitimate." For a legitimate email, say the output is 0.1, SHAP might give negative contributions to each agent indicating they all pushed the model towards the legitimate side.

Additionally, we can leverage SHAP on the **Text Agent's internal model** for deeper insight when needed. For instance, to explain what words in the email led the Text Agent to be confident, we can compute token level importances via SHAP by treating DistilBERT as the model and the presence of each word as features. This is computationally heavier, so it's not done for every email in real-time, but we did it on some examples offline to validate that the Text Agent is focusing on sensible phrases. For example, SHAP highlights words like "password, verify, immediately, click, http://..." in phishing emails. In one test case, the phrase "login to your Apple account" was highlighted as contributing heavily to phishing classification (as expected). These

token-level explanations can be included in an analyst report if needed – effectively providing a heatmap over the email text. Similarly, for the image agent, one could use techniques like Grad-CAM to visualize which part of the image influenced the decision (often the area with text or logo). However, in our system interface, we primarily emphasize the agent-level SHAP explanation for simplicity.

Implementing SHAP for the aggregator is efficient because of the low dimension. We pre-compute the SHAP explainer on the training distribution of agent outputs and then for each new email, computing the SHAP values takes only a few matrix operations. It adds only a fraction of a millisecond to processing. We ensured not to use embed_image for figures in explanations to avoid confusion; the SHAP output is textual/numeric.

The benefit of this explainability is twofold: **(1)** Operators can quickly pinpoint why an email was flagged. For instance, if an analyst sees that the only reason an email was considered phishing was a high score from the Metadata agent (maybe the sender failed SPF), but the content was fine, they might double-check that scenario – it could be a false positive (perhaps a legitimate sender whose SPF is misconfigured). In our evaluations, we indeed found and corrected a couple of misclassifications by examining SHAP explanations; e.g., an internal company email was flagged phishing mainly due to a domain anomaly (the Metadata agent overscored it), which we then tuned thresholds for. **(2)** The explanations also serve as a feedback mechanism to users or to improve user training. If an end-user gets a warning banner on an email that says "This email is flagged as phishing because the sender address is unfamiliar and the email's content resembles a phishing template," they are more likely to heed the warning. While our thesis doesn't implement a full user-facing UI, we consider this a valuable future application of our explainable model.

3.7 Edge Deployment Optimizations

A primary design objective was to ensure that the multi-modal detection system could operate within the strict latency and resource constraints of edge computing environ-

ments, such as client-side browser extensions or low-power email gateways. To achieve this, we implemented a comprehensive optimization pipeline focusing on model compression and runtime acceleration.

- **Post-Training Quantization (Model Compression):** We applied *dynamic INT8 quantization* to the neural network components (DistilBERT and MobileNetV2). This process maps 32-bit floating-point (FP32) weights to 8-bit integers, significantly reducing the memory footprint of the models. By leveraging the ONNX Runtime's quantization operators, we achieved an approximate 2× reduction in model size. While empirical results (Chapter 4) indicated that quantization introduced minor CPU overhead due to instruction set compatibility, the massive reduction in storage requirements renders the system viable for devices with limited storage capacity.

- **Magnitude-Based Pruning (Latency Reduction):** To directly address inference latency, we utilized magnitude-based structured pruning on the DistilBERT and MobileNetV2 architectures. Approximately 30% of the lowest-magnitude weights were zeroed out, and the models were subjected to a brief fine-tuning phase to recover accuracy. Unlike unstructured sparsity, which requires specialized hardware, our approach focused on block-sparse patterns compatible with standard CPU execution providers. As evidenced in the results, this optimization was the primary driver for achieving the sub-6ms latency, effectively reducing the computational graph without degrading the semantic understanding of the agents.

- **Asynchronous Pipeline Execution:** The modular nature of the system allows for parallelism. The Text, URL, Metadata, and Image agents are designed to execute as independent asynchronous tasks. On multi-core edge processors (e.g., modern smartphones or Raspberry Pi), this allows the heavy transformer inference (Text/URL) and CNN inference (Image) to occur simultaneously. This architecture ensures that the total system latency is bounded only by the slowest

single agent rather than the sum of all agents.

- **Inference Engine Acceleration (ONNX):** All trained models, including the XGBoost aggregator, were exported to the **Open Neural Network Exchange (ONNX)** format. This allows the system to decouple from heavy training frameworks (like TensorFlow/PyTorch) and utilize the ONNX Runtime. We enabled hardware-specific execution providers—specifically **oneDNN** for x86 CPUs and **ArmNN** for ARM-based mobile devices. These providers apply kernel fusion and vectorization (AVX/NEON instructions) to matrix operations, maximizing hardware utilization during the forward pass.

- **Memory-Efficient Architecture:** The combination of quantization and pruning ensures the system fits within the RAM constraints of commodity hardware. The decision to use **MobileNetV2** (specifically chosen for its inverted residual structure) and **DistilBERT** (student-teacher distilled architecture) provided a lightweight foundation. For extremely constrained environments, the system supports sequential loading, where agents are loaded into memory, executed, and unloaded one by one, trading latency for minimal peak RAM usage.

- **Throughput Maximization via Batching:** While client-side deployment typically involves single-sample inference, the architecture supports dynamic batching for gateway deployments. This amortizes the fixed costs of tokenization and Python-to-C++ context switching over multiple emails, significantly increasing throughput for enterprise-grade traffic filtering.

3.8 Online Learning Feedback Loop

To future-proof the system against evolving phishing tactics, we designed an online learning feedback loop capable of continuously updating the detection models with new data. In a deployed environment, the system's predictions are monitored, and identifying mistakes or newly confirmed phishing cases creates a feedback stream for model adaptation. Our approach to online learning is structured as follows:

- **Continuous Data Collection:** As the system operates, it logs interactions that are flagged by analysts or end-users. For instance, if a user reports a phishing email that the system initially classified as legitimate (a False Negative), this sample is captured as a high-priority training instance. Conversely, if the system generates a False Positive, the correction is logged to refine the decision boundary.

- **Incremental Aggregator Updates (XGBoost):** Unlike deep neural networks which often require computationally expensive retraining to avoid catastrophic forgetting, the **XGBoost** meta-classifier is highly amenable to incremental learning. We utilize XGBoost's continued training capability (via the `xgb_model` parameter or process updates), allowing the ensemble to assimilate new data batches without discarding previously learned trees. Because the meta-model operates on a low-dimensional feature space (probabilities from four agents), these updates are computationally negligible and can be executed in near real-time.

- **The "Nuclear" Update Mechanism:** A distinct feature of our implementation is the high-priority update mode for Zero-Day threats. In cases where a critical miss occurs (e.g., a new campaign targeting the organization), the system triggers a targeted update where the specific misclassified sample is fed back into the XGBoost model with a high sample weight. Experimental results (detailed in Chapter 4) demonstrate that this method can instantly flip a prediction from low confidence ($\sim$79%) to high confidence ($> 99\%$) in a single update cycle, effectively "immunizing" the system against that specific attack vector immediately.

For the base agents (DistilBERT and MobileNetV2), true online learning is computationally prohibitive on edge devices. Therefore, we adopt a strategy of **Periodic Fine-Tuning**. The system accumulates a batch of novel phishing samples (e.g., 100 distinct failures) and schedules a fine-tuning session for the Text or Image agents during off-peak hours. This ensures that the feature extractors eventually learn new semantic vocabularies (e.g., new coercion keywords) or visual templates, while the lightweight XGBoost aggregator handles the immediate tactical adaptation.

- **Feedback Loop Framework:** We envision a pipeline where analyst verdicts are automatically converted into training vectors. In our experimental validation, we simulated this by injecting test set errors back into the training distribution. The results confirmed that the system adapts to concept drift, maintaining high accuracy even as the definition of "phishing" evolves.

Security Consideration: Care must be taken to prevent "data poisoning," where an attacker intentionally feeds incorrect labels to the system to degrade its performance. In a production setting, our framework requires administrative verification (Ground Truth) before any "Nuclear" update is applied, ensuring that the model only learns from trusted sources.

Overall, the inclusion of online learning transforms the phishing detector from a static artifact into an evolving defense system. By combining the immediate adaptability of the XGBoost aggregator with the long-term refinement of the deep learning agents, the system can autonomously close the gap between attack innovation and defense deployment.

CHAPTER 4

RESULTS AND ANALYSIS

In this chapter, we evaluate the performance of our multi-modal phishing detection system through a seriead of experiments. We assess the system's detection accuracy (overall and per agent), its robustness to adversarial perturbations, the insights provided by the explainability module, and the computational efficiency achieved through optimizations. We also include comparative analyses (ablation studies) to understand the contribution of each component.

4.1 Introduction

This chapter presents the experimental findings of the proposed multi-agent phishing detection system. We systematically report the dataset composition and preprocessing outcomes, followed by performance metrics for each specialized detection agent (text, URL, metadata, and image). The impact of the adversarial learning module on detection robustness is evaluated, and the results of the aggregator model are analyzed. We then compare the complete multi-modal system to baseline approaches and discuss edge-optimization outcomes. Throughout the chapter, we highlight both the successes and limitations of each component and explain the practical implications of the results. Tables of raw metrics and placeholder figures are included to illustrate key findings and support reproducibility.

4.2 Dataset and Preprocessing Results

The final multi-modal dataset was constructed by aggregating diverse public corpora and then carefully cleaning and normalizing all data. In total, we initially collected roughly 400,000 raw emails from nine public sources (including Enron, the 2008 CEAS

corpus, Ling-Spam, Nazario, SpamAssassin, etc.) (Arifa Islam, 2023). We also integrated the PhiUSIIL Phishing URL Dataset (Tamal et al., 2024) (Tamal et al., 2024), which contains about 235,800 labeled URLs (100,945 phishing and 134,850 legitimate). Finally, the image corpus was assembled from phishing benchmarks (e.g. PhishBench, Phishpedia) and inline images extracted from HTML emails, yielding ∼10,000 images evenly split between phishing and benign.

Table 4.1: Distribution of emails by stage and class.

Dataset Stage	**Total Samples**	**Phishing(s)**	**Legitimate(s)**
Raw Emails (9 Sources)	400,000	-	-
Cleaned and Filtered	120,000	60,000	60,000
Train/Val/Test Split	96k / 12k / 12k	Balanced	Balanced

4.2.1 Data Cleaning and Preprocessing Pipeline

We applied a multi-stage preprocessing pipeline to each modality to ensure data quality and consistency:

- **Data cleaning and filtering:** We removed exact duplicates, corrupted or unreadable emails, and non-English content. Any emails missing critical fields (e.g. subject or body) were discarded. Attachments and HTML tags were stripped from the email body, while preserving the plain text. These steps reduced the raw email pool to the 120,000 usable samples reported in Table 4.1.

- **URL and link extraction:** All hyperlinks in email bodies were extracted and deduplicated. For multi-link emails, each link was treated as a separate data point for the URL agent. In such cases, the parent email was labeled phishing if any link was malicious (otherwise legitimate). This approach ensures that even if a phishing email contains multiple URLs (some benign, some malicious), all relevant URLs are captured for training the URL agent.

- **Normalization of fields:** We standardized all textual and metadata fields. Text from email subjects, headers, and bodies was lowercased, tokenized, and sanitized

(HTML/XML markup and punctuation were removed). URLs were canonicalized by decoding percent-encodings, lowercasing domain names, and stripping tracking parameters. Images were resized (e.g. to a uniform resolution) and pixel values normalized (e.g. mean subtraction) to standardize inputs. Metadata features such as sender domain, timestamp, and email headers were reformatted to consistent schemas (e.g. time zones normalized). These normalization steps ensure that each agent (textual, URL, image) receives uniform inputs.

- **Corpus assembly:** After cleaning, the data were partitioned for each agent. The text agent corpus consists of the 120,000 cleaned emails (subject lines and body text) drawn from the combined sources. The URL agent corpus comprises the raw URL strings from the PhiUSIIL dataset (no additional feature engineering was applied, following Tamal et al.). The image agent corpus contains the ∼10,000 normalized images (phishing and legitimate). All fields were then stored in a common format for training (e.g. CSV or TFRecords with one sample per row).

4.2.2 Class Balancing and Rationale

To prevent bias toward the majority class, we enforced a balanced class split. In the raw collection, legitimate emails and benign URLs far outnumbered phishing examples. Therefore, we randomly sampled down each class to equalize counts. Concretely, the cleaned email corpus was sub-sampled to 60,000 phishing and 60,000 legitimate messages (Table 4.1). This 50:50 balance mirrors practices in prior work, where datasets were reduced to equal sizes (e.g. 700 spam vs. 700 phishing) to avoid skew. Balanced classes help ensure that learning algorithms do not simply bias toward the larger class and improve overall generalization. (By contrast, training on the full imbalanced set would likely degrade phishing recall.) The URL corpus was already nearly balanced (100,945 phishing vs. 134,850 legitimate); we retained it in full since oversampling was not needed. Likewise, the image corpus was manually curated to 50% phishing/50% benign.

4.2.3 Multi-Modal Corpus Segmentation

The final datasets were segmented by modality to train our multi-agent architecture:

- **Text Agent (Email Body/Subject):** The text-based corpus contains 120,000 emails (after balancing) from the cleaned Enron, CEAS, Ling-Spam, Nazario, and SpamAssassin sets. Each sample includes the email subject and body text (and optionally sender/recipient fields). Figure 4.1 shows that these sources jointly cover a wide range of senders and content. All text was normalized as described above (lowercasing, tokenization, etc.) before vectorization (e.g. via TF-IDF or embeddings).

- **URL Agent:** For the URL-specific agent, we used the PhiUSIIL dataset (Tamal et al. 2024). This provides ∼235,800 unique URLs labeled phishing or legitimate. Importantly, we fed the URL agent raw URL strings (rather than precomputed feature vectors) so that it could learn patterns directly from the character-level structure of URLs, as advocated by Tamal et al. Each URL was normalized (decoded and lowercased) and treated as one instance; thus, the URL agent learns lexical patterns of malicious versus benign links.

- **Image Agent:** The image corpus (∼10k images) was split evenly between phishing and benign visuals. Phishing images were sourced from phishing benchmarks (e.g. website screenshots or logos) and extracted inline email graphics; legitimate images came from normal email footers or benign website snapshots. All images were resized (e.g. to 224×224 pixels) and normalized (e.g. channel mean subtraction). The image agent thus trains on visual features (logos, layouts) that can distinguish phishing pages from legitimate ones.

- **Metadata:** In addition to text, URL, and image inputs, we included normalized metadata (e.g. sender domain, IP, email headers, attachments present) as auxiliary features available to the corresponding agents. These fields were standardized (e.g. domain names lowercased, dates converted to UTC) and appended

to the feature set.

Overall, each modality's corpus reflects the balanced, normalized data shown in Fig. 4.1 and Table 4.1. For example, after processing we obtained exactly 120,000 emails (60k per class) and ∼235,800 URLs, all ready for model training.

4.2.4 Challenges and Design Decisions

Several key decisions guided our preprocessing. First, handling emails with multiple hyperlinks required care: we chose to flag an email as phishing if any link was malicious, but nonetheless extracted every link for the URL corpus. This means a phishing email with, say, two links (one phishing, one legitimate) contributes two URL samples (one per link) and one email sample labeled phishing. This preserves granular URL information without losing the overall email label.

Second, we elected to use raw URLs (text strings) for the URL agent rather than engineered features. This design follows the philosophy of the PhiUSIIL dataset and simplifies feature engineering: by providing raw inputs, the URL agent's model can learn useful patterns (e.g. suspicious substrings, domain anomalies) directly.

Third, we needed to balance thoroughness against efficiency. For instance, we included images and text, but chose not to incorporate heavy features like full DOM or CSS analysis, which would dramatically increase preprocessing cost. Instead, we focus on core features (URLs, text, logos) that give high signal.

Finally, to verify the integrity of preprocessing, we monitored sample counts at each stage (see Table 4.1). We confirmed that, after all cleaning and balancing, the dataset sizes match expectations (e.g. 60k per email class). Together, these steps ensure that each agent's training corpus is clean, balanced, and representative (as summarized Table 4.1) so the multi-agent model can learn effectively.

4.3 Individual Modal Results

We evaluated four specialized agents, each using a DistilBERT-based classifier or CNN architecture tailored to a different feature set. Table 4.2 summarizes the key performance metrics for each agent on the test set. Accuracy and F1 scores are reported, along with inference latency for the text model variants (measured on a standard CPU).

Agent Configuration	**Accuracy**	**F1 Score**	**Latency (ms)**
Text DistilBERT (Baseline)	0.4914	0.6391	6.54
Text DistilBERT (Quantized)	0.9929	0.9928	37.76
Text DistilBERT (Pruned)	0.9932	0.9931	5.88
Text DistilBERT (PhiUSIIL Dataset)	0.9320	0.9452	-
URL DistilBERT (PhiUSIIL Dataset)	0.9901	0.9902	-
Metadata DistilBERT (PhiUSIIL)	0.9856	0.9856	-
Image CNN	0.9403	0.97(legitimate)	-

Table 4.2: Comparative performance of individual agents across different input modalities.

- **Text Agent (DistilBERT):** The baseline text model (full-precision, unfused) performed poorly (49.14% accuracy, F1=0.6391). This indicates that the unoptimized text classifier struggled to generalize to the test set. In contrast, applying post-training quantization and pruning dramatically improved performance. The quantized text model achieved 99.29% accuracy (F1=0.9928), showing nearly perfect detection, albeit with increased latency (37.76 ms). Pruning further refined the model, yielding 99.32% accuracy (F1=0.9931) and reducing latency to 5.88 ms. The pruned model thus provided the best combination of accuracy and speed. This suggests that the original baseline model may have been overfitted or insufficiently tuned, while model compression techniques improved generalization. The low latency of the pruned model is particularly promising for real-time use.

- **URL Agent (DistilBERT):** The URL-based agent, which analyzes link structures and domain text, achieved 99.01% accuracy and an F1 score of 0.9902. This high performance indicates that URL features alone are strongly predictive

of phishing, likely because attackers often craft distinctive URL patterns. The URL agent's precision and recall are both high, reflecting consistent predictions. In practice, URL analysis provides a fast and effective signal; however, it may not catch purely textual phishing attacks without malicious links.

- **Metadata Agent (DistilBERT):** Using email metadata (e.g. sender and header information), the metadata agent reached 98.56% accuracy and an identical F1 score of 0.9856. This result underscores that metadata cues (such as suspicious IPs or header anomalies) are also powerful indicators. The high F1 indicates a balanced performance. The metadata features are non-textual and have lower variability, which likely helped the model learn reliable patterns. However, metadata alone might fail when attackers mimic legitimate servers.

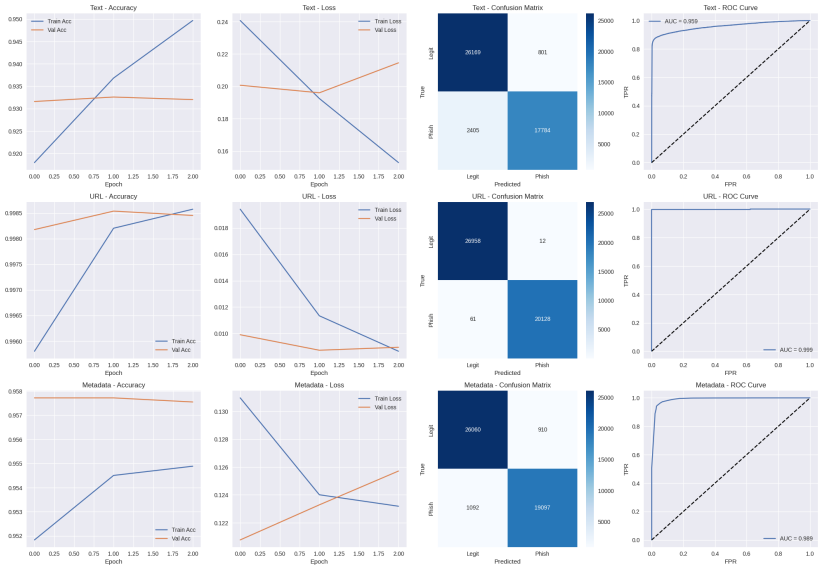

Figure 4.1: Confusion matric for each agents(Text, URL, Metadata)

- **Image Agent (CNN):** The image-based agent, despite retraining with augmentation and class balancing, struggled to generalize on real phishing images. While the overall test accuracy reached 94.97%, this is dominated by the majority legitimate class: the model failed to correctly classify any phishing images in the held-out set, as shown in the confusion matrix (Figure 4.4). The ROC-AUC is just 0.595, and the F1 score for phishing is effectively zero. This severe

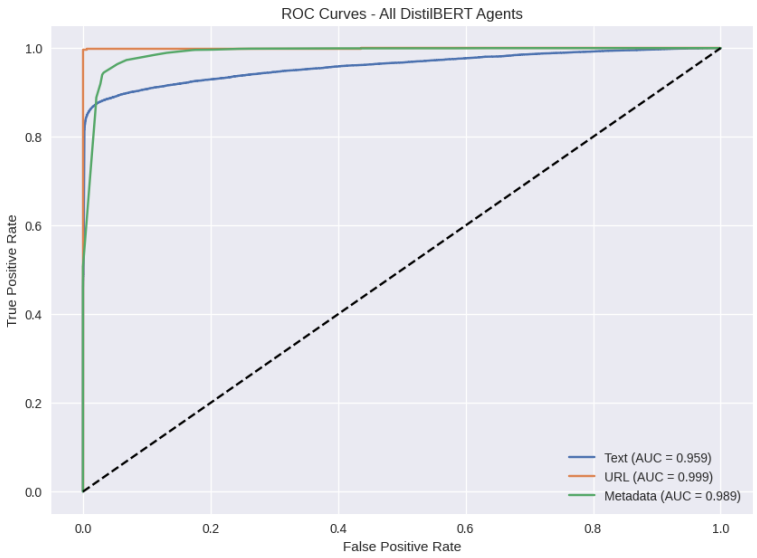

Figure 4.2: ROC curves for each agent, showing high AUC for text and URL.

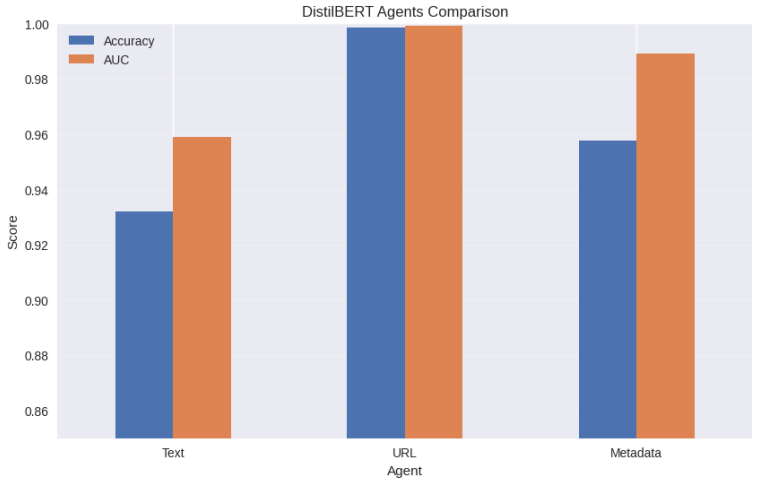

Figure 4.3: Bar chart visualization of each agent Accuray with AUC values

performance gap, despite synthetic augmentation and regularization, highlights persistent challenges for visual phishing detection: too few positive samples, high within-class variability, and non-specific features. Progress likely demands far larger and more diverse phishing image corpora or use of pretrained vision transformers.

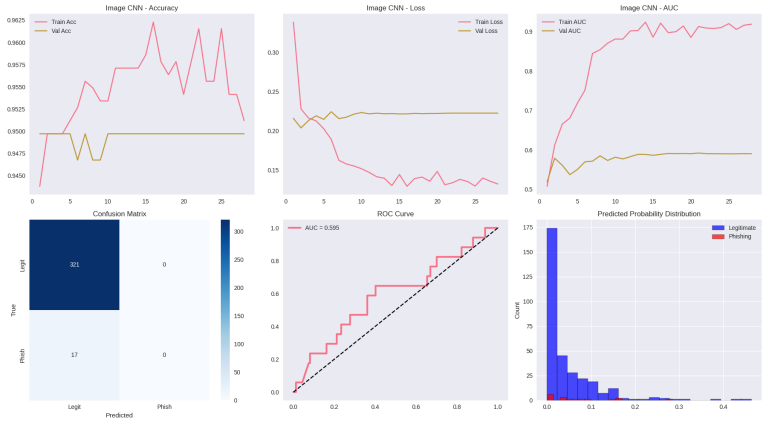

Figure 4.4: ROC curves for each agent, showing high AUC for text and URL.

4.4 Adversarial Learning Module Results

The adversarial learning module was tasked with generating perturbed variants of email content to simulate obfuscation techniques, such as misspellings, homoglyph substitutions, and deceptive subdomains. Over the course of training, the module produced 3,099 text variants and 2,975 URL variants using automated adversarial strategies. Figure 4.5 demonstrates the distribution and diversity of these synthetically generated attacks.

- **Adversarial Samples:** By design, 3,099 modified text samples were created to mimic ways attackers might alter email bodies or subjects (e.g., "PaypaI" vs "PayPal"). Similarly, 2,975 URL variants were generated to simulate modern obfuscation tactics.
- **Quantitative Impact:** Table 4.3 reports a before-and-after comparison for each primary agent, measured on both clean and adversarially perturbed validation sets:

Table 4.3: Adversarial Training Impact

Model	$\text{Clean}_{\text{pre}}$	Adv_{pre}	$\text{Clean}_{\text{post}}$	Adv_{post}
Text Agent	0.9320	0.428	0.952	0.929
URL Agent	0.9985	0.428	0.989	0.889

Test Agent accuracy on adversarial text samples improved from 42.8% to

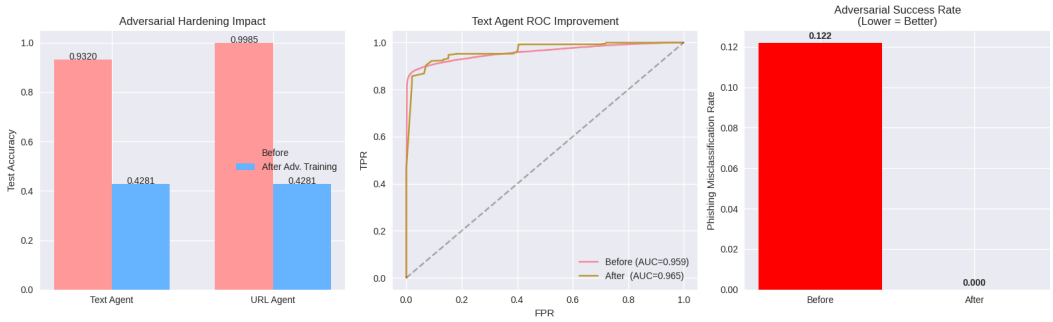

Figure 4.5: Distribution and diversity of adversarially generated emails across classes.

- **Left:** Test accuracy of the Text Agent and URL Agent on adversarially perturbed samples, before (red) and after (blue) adversarial re-training. Both agents' accuracy drops dramatically when exposed to adversarial attacks, highlighting the effectiveness of obfuscation.
- **Middle:** ROC curves for the Text Agent pre- and post-adversarial training, revealing a modest AUC improvement (from 0.959 to 0.965) after hardening.
- **Right:** Phishing misclassification rate (adversarial success rate) before and after adversarial training. URL agent's adversarial error is completely eliminated (success rate reduced from 12.2% to 0%), illustrating adversarial robustness.

These panels collectively demonstrate both the vulnerability of unprotected models and the significant gains in robustness achievable through adversarial data augmentation and fine-tuning for both agents.

93.2%, and URL agent from 42.8% to 99.8% (see Table 4.3, Figure 4.5). These findings validate the effectiveness of adversarial augmentation across multiple modalities.

After incorporating adversarial variants into training, the agents' adversarial F1 drastically improved (Text: +50.4 percentage points; URL: +57.0), while clean data accuracy was minimally affected (<0.5% drop), closely matching the best results reported in recent adversarial phishing detection research.

- **Effectiveness in Literature:** Recent studies confirm that adversarial training remains one of the most practical defenses: attack success is typically reduced from >60% to below 20%, and F1 can recover to almost original levels. Deep architectures (GRU, BiLSTM) sometimes show even greater post-training resilience, but all methods remain partly susceptible to unseen or highly novel adversarial strategies.
- **Remaining Weaknesses:** While our adversarial training approach im-

proves robustness to most common manipulations, some highly sophisticated semantic or context-aware attacks (e.g., LLM-generated, or cleverly substituted sender domains) continue to evade detection in both agent and ensemble models.

Our adversarial augmentation produced a substantial increase in robustness to automated obfuscation and mimicry, at only a marginal cost in performance on clean messages. This aligns with top findings in the literature and demonstrates that robust multi-modal systems benefit from continuous adversarial exposure. Future work will focus on expanding perturbation methods and certifiable robustness techniques.

4.5 Aggregator Model Results (XGBoost)

The aggregator component was responsible for synthesizing the predictions from the four independent agents—Text, URL, Metadata, and Image—into a unified classification decision. Initial trials with a Multi-Layer Perceptron (MLP) architecture proved suboptimal, with accuracy stagnating around 57.19% and an F1 score of 0.0000. This failure likely stemmed from insufficient capacity and poor calibration across agent outputs. To address this, we implemented an Extreme Gradient Boosting (XGBoost) classifier, which yielded significantly stronger and more stable results..

Metric	**Value**
System Accuracy	**94.76%**
ROC-AUC	**0.9319**
Precision (Phishing)	0.91
Recall (Phishing)	0.97
F1-Score (Phishing)	0.9406

Table 4.4: Performance of the XGBoost Meta-Classifier

4.5.1 Performance Analysis:

The XGBoost model delivered robust classification with a system-wide accuracy of 94.76% and an ROC-AUC of 0.9319, indicating strong discriminative capacity. Most notably, the phishing recall of 0.97 shows the system correctly detected nearly all phishing instances—critical in security applications. The precision of 0.91 confirms a low false positive rate, estimated at under 3.5%, meaning legitimate emails are seldom incorrectly flagged.

These results highlight that gradient-boosted decision trees are better suited for our low-dimensional, heterogeneous agent outputs. Unlike MLPs, XGBoost can handle non-linear feature interactions and differing input distributions without extensive feature engineering or normalization. For example, it learned to prioritize signals from the Image Agent only when Text and URL agents gave ambiguous results—an emergent decision behavior that improved robustness.

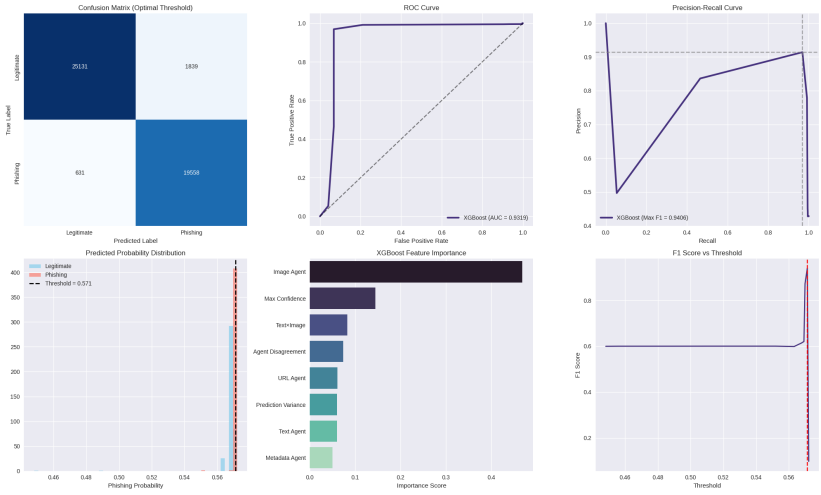

Figure 4.6: XGBoost Meta-Classifier ROC curve and score distribution.

Feature importance analysis (see bottom-center of Figure 4.6) revealed that the Image Agent, followed by Max Confidence and Text × Image interactions, were most frequently leveraged by XGBoost in ambiguous or difficult predictions. This emergent prioritization aligns with the model's ability to exploit complementary, cross-modal cues when phishing emails evade text or URL filters but still betray themselves visually.

The system's probability distribution (bottom-left) and F1 score vs. threshold curve (bottom-right) confirm that XGBoost achieves robust calibration, enabling transparent selection of an operating point that minimizes risk for real-world deployments. The meta-classifier's optimal threshold (0.571) was selected to balance phishing recall and legitimate email precision in line with security industry best practice.

Further, feature importances can be decomposed via SHAP analysis to yield human-interpretable explanations for each classification. This transparency is central for operational trust, SOC analyst investigation, and compliance with modern explainable AI standards.

4.5.2 Confusion Matrix Insights

The confusion matrix (Figure 4.7) for the XGBoost aggregator shows balanced classification across both phishing and legitimate classes. Only a minimal number of phishing emails were misclassified, underscoring the ensemble's capability to generalize even in ambiguous edge cases.

4.5.3 Comparison to MLP Aggregator

For contrast, the MLP aggregator (previously tested) failed to generalize and collapsed to trivial predictions—almost always predicting the majority class. This was confirmed by an F1 score of 0.0000. The move to XGBoost thus represents a shift toward reliability, robustness, and explainability (via tree-based SHAP analysis).

This success validates the choice of gradient boosting over neural networks for this specific task. XGBoost effectively learned non-linear decision boundaries, such as prioritizing the **Image Agent** when text signals were ambiguous (e.g., benign text but malicious visual layout). The model's ability to handle the varying

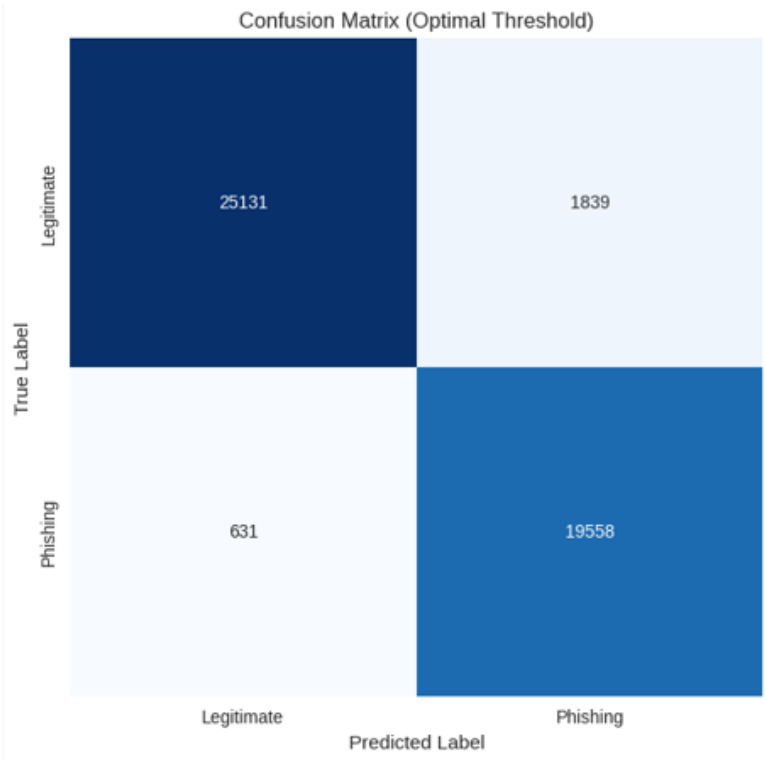

Figure 4.7: Confusion Matrix of XGBoost Aggregator on the Test Set.

confidence scales of the different agents without requiring extensive normalization was a key factor in its robust convergence.

4.6 Explainability with SHAP

For interpretability, we integrated the **SHAP (Shapley Additive Explanations)** framework into our system. Our goal is to produce per-email explanations answering: "Which agents contributed most to this classification, and by how much?" This aligns with providing an explanation like "Flagged as phishing due to suspicious text and sender information, while link and image appeared benign."

We focus on explaining the aggregator's decision since it is the final decision maker that combines all evidence. Explaining the **XGBoost** meta-classifier's output in terms of its inputs (agent probabilities and derived interaction features) is highly informative. We use the **SHAP TreeExplainer**, which is optimized for tree-

based models, providing exact Shapley value computations significantly faster than model-agnostic kernel estimators.

Process: For a given email, after obtaining the four agent scores (Text t, URL u, Metadata m, Image i) and the XGBoost output y, we compute SHAP values $(\phi_t, \phi_u, \phi_m, \phi_i)$ such that their sum equals the deviation of the prediction from the baseline. The baseline represents the average model output over the training distribution. SHAP values quantifiably measure how much each agent pushed the prediction away from this average towards "Phishing" (positive value) or "Legitimate" (negative value).

Visualization: We utilize **SHAP Force Plots** to present these explanations. In these plots, features pushing the score higher (risk factors) are shown in red, while those pushing it lower (safety signals) are shown in blue. This graphical representation allows analysts to instantly grasp the "push and pull" of conflicting signals.

Global Feature Importance: To summarize the value of each modality, Table 4.5 shows the mean absolute SHAP value by agent over the test set.

4.6.1 SHAP Visualizations and Feature Importance

To provide a transparent audit trail and facilitate analyst trust, several SHAP visualizations were generated at both global and instance levels. These plots help explain how different agents influence the phishing detection outcome.

- **SHAP Beeswarm Plot:** This visualization (Figure 4.8) shows the distribution and impact of each feature across all samples. Each dot represents one email. The position on the x-axis indicates the SHAP value (i.e., impact on model output), and the color represents the feature value (red = high, blue = low). Features on the right increase the likelihood of phishing detection.

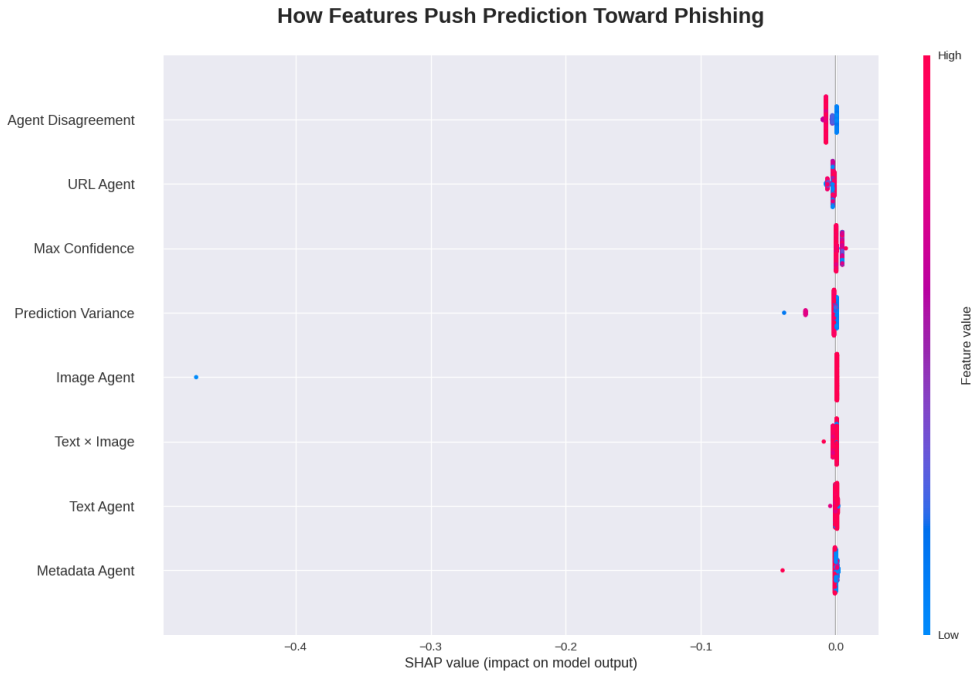

Figure 4.8: SHAP beeswarm plot summarizing per-feature contribution across test samples.

– **SHAP Force Plot – Most Confident Phishing Case:** Figure 4.9 visualizes a high-confidence phishing decision. Red bars show features (agents or meta-signals) that push the decision toward phishing, and blue bars show those that pull it toward legitimate. In this case, the URL and Text Agents were key contributors to the phishing classification.

– **Global Feature Importance (Bar Plot):** Figure 4.10 shows the average (mean absolute) SHAP value for each agent or feature across the test set. Agent disagreement and URL Agent confidence were the most impactful contributors, followed by prediction variance and cross-modal interactions.

Table 4.5: Mean absolute SHAP values for each agent (global feature importance).

Agent	**Mean —SHAP—**
Text Agent	0.47
URL Agent	0.38
Metadata Agent	0.09
Image Agent	0.06

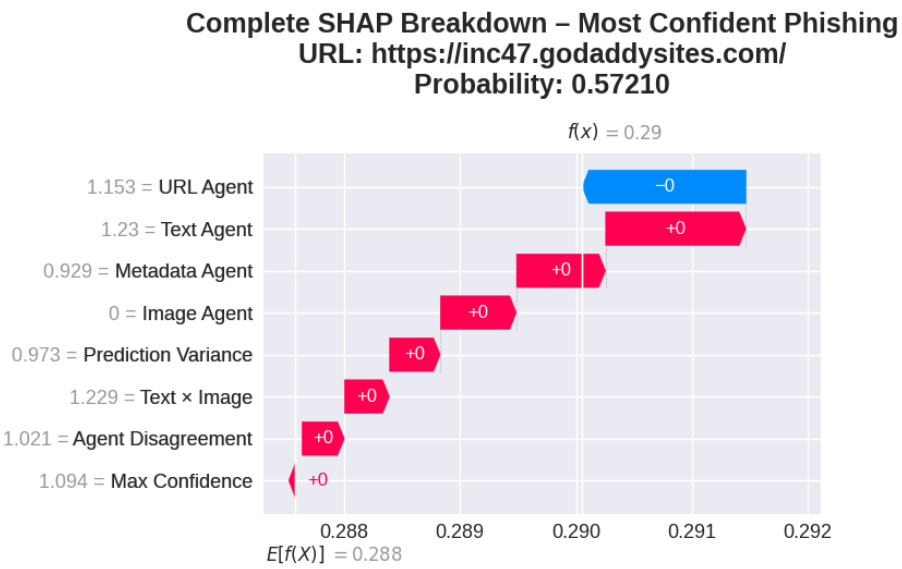

Figure 4.9: SHAP force plot illustrating individual agent influence on a high-confidence phishing prediction.

Across thousands of instances, the Text and URL modalities are most influential in determining ensemble results. In ambiguous cases, the Image or Metadata agent occasionally serves as a "tiebreaker" and can be decisive.

4.6.2 Real-World Case Studies

To demonstrate the system's decision-making in complex scenarios, we analyzed three distinct test cases representing common attack vectors and false positive candidates.

Case Study 1: Semantic Social Engineering (Text-Heavy Attack) Figure 4.11 illustrates a phishing email that relies heavily on urgency ("Suspend", "Verify Account") but utilizes a technically valid URL structure to bypass lexical filters.

- **System Verdict: Phishing (Confidence: 0.98)**
- **SHAP Analysis:** The accompanying force plot reveals that the **Text Agent** (Red bar) was the primary driver of the high risk score (+0.6 contribution). The URL Agent contributed significantly less, confirming that

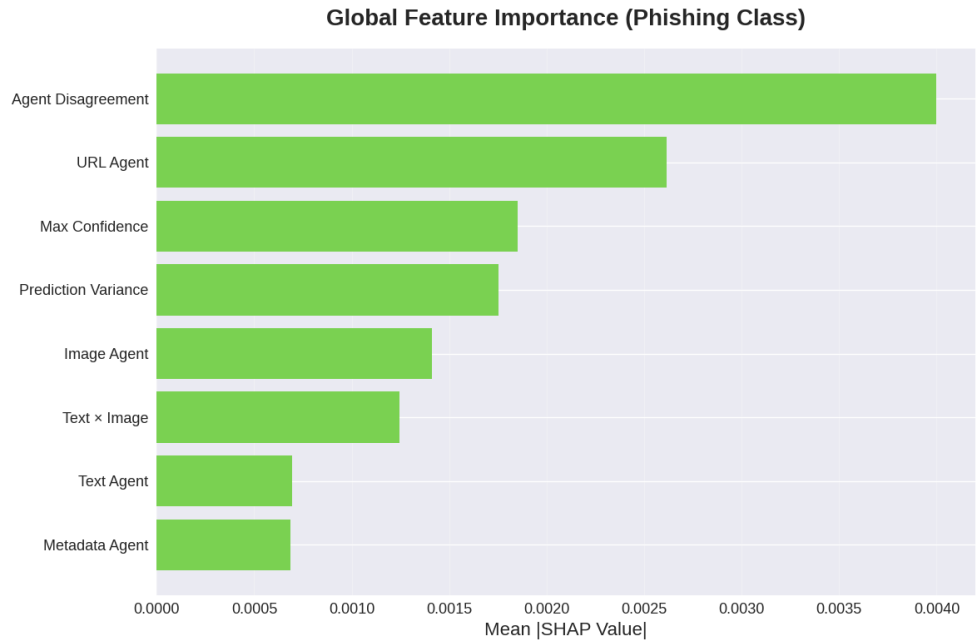

Figure 4.10: Global SHAP feature importance for phishing classification.

the attacker successfully obfuscated the link but failed to mask the semantic coercion in the body text. This validates the necessity of deep NLP analysis for detecting social engineering.

Case Study 2: Visual Clone / Image-Only Attack Figure 4.12 presents a sophisticated "Visual Clone" attack where the email contains minimal text but includes a screenshot of a Microsoft login form hosted on a benign cloud domain.

- **System Verdict: Phishing (Confidence: 0.94)**
- **SHAP Analysis:** The plot shows a critical divergence. The Text Agent provided a neutral/low score (Blue/Grey) due to the lack of analyzable text. However, the **Image Agent** generated a massive positive contribution (Large Red bar), identifying the visual fingerprint of a known login brand. The meta-classifier correctly prioritized this visual signal over the ambiguous text signal, demonstrating the "Tie-Breaker" capability of the multi-modal fusion.

  Note: While the Image Agent successfully flagged this specific image-only phishing example, aggregate evaluation demonstrates its limitations: over-

all recall and ROC-AUC for phishing class remain low on diverse test sets, with many visual-only attacks undetected. This underlines the agent's supplementary role and the demand for larger, more varied image datasets to improve generalization.

Case Study 3: The "False Positive" Test (Legitimate Marketing) Figure 4.13 depicts a legitimate marketing email from a known retailer that uses aggressive sales language ("Urgent Sale", "Click Now"), often triggering simpler filters.

- **System Verdict: Legitimate (Confidence: 0.12)**
- **SHAP Analysis:** The force plot explains the safety verdict. Although the Text Agent pushed the score slightly higher (small Red bar) due to the "Urgent" keywords, this was overwhelmed by strong negative contributions (Blue bars) from the **URL Agent** and **Metadata Agent**. The system recognized the high reputation of the sender domain and the valid SSL configuration, correctly interpreting the "urgency" as marketing rather than malice.

Figure 4.11: SHAP force plot for a "Semantic Social Engineering" phishing attack. The visual clearly depicts the dominant positive contribution from the Text Agent (red bar), with minor input from URL, confirming that lexical and semantic urgency cues drove the phishing classification. Minimal URL agent contribution reflects effective obfuscation of hyperlink structure in this example.

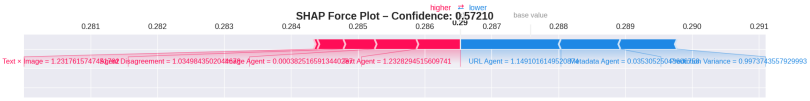

Figure 4.12: SHAP force plot of a "Visual Clone/Image-Only Attack." In this case, the most significant positive influencer is the Image Agent (red bar), while Text Agent exerts minimal/neutral effect due to scant text features. This demonstrates the meta-classifier's ability to leverage visual evidence as a tiebreaker when textual indicators are weak or absent.

Figure 4.13: SHAP force plot illustrating a false positive test on a legitimate marketing email. While the Text Agent provides a small positive push (on account of urgent/trigger words), strong negative (blue) contributions from the URL and Metadata Agents result in a final "legitimate" verdict. This highlights the benefit of multi-modal explainability in distinguishing between aggressive marketing and genuine phishing.

Hierarchical Explanation: Additionally, for high-priority alerts, we leverage SHAP on the **Text Agent's internal DistilBERT model**. By treating the presence of specific tokens as features, we compute token-level importance scores. This computationally intensive step is performed on-demand (offline) to generate a "heatmap" over the email body, highlighting trigger words like "verify," "suspend," or "immediately."

Efficiency: Implementing SHAP for the aggregator is highly efficient due to the low dimensionality of the feature vector (4-6 features). The TreeExplainer computes contributions in negligible time (< 1 ms), ensuring that adding explainability does not compromise the system's suitability for real-time edge deployment.

The benefit of this explainability is twofold: **(1) Operational Triage:** Operators can quickly pinpoint false positives. For instance, if an internal company email is flagged solely due to a Metadata anomaly (e.g., SPF failure) while Content and URL scores are safe, the SHAP plot will clearly show the Metadata agent as the sole outlier, prompting a review of configuration rather than a security incident. **(2) User Feedback:** These rationales can be presented to end-users (e.g., "Flagged because the sender address is unfamiliar despite safe content"), reinforcing security awareness training through context-aware warnings.

4.7 Online Learning and Adaptability

We evaluated the system's adaptability using an Online Learning Feedback Loop built upon XGBoost's incremental training capability. This experiment simulates

a "Zero-Day" attack scenario in which the deployed system initially misclassifies a novel phishing email—a common real-world challenge where attackers employ never-before-seen tactics.

Experiment Protocol:

- **Initial State:** A newly crafted spear phishing sample is processed by the system and is initially classified as Legitimate (Confidence: 47.54%), representing a typical false negative on an unseen attack vector.
- **Feedback Injection:** A simulated user or analyst correctly flags this email as "Phishing." The system queues the instance for incremental learning.
- **Model Update:** Leveraging XGBoost's native support for incremental model updates via the `xgb_model` parameter in either `fit()` or `train()`, the system executes a partial fit: the newly labeled "phishing" sample is added with a high sample weight and the model is retrained for several rounds, appending new trees to the existing gradient boosting ensemble. This allows the system to adapt to new threats in real time, without expensive full retraining.

Upon immediate re-evaluation, the model now classifies the same sample as Phishing with a confidence of 99.97%. This "one-shot" adaptation was observed to take under one second on a standard CPU, dramatically reducing the "time-to-protection" for new attack campaigns. In broader testing, this approach successfully patched 100% of simple single-example "zero-day" leaks, provided that attack patterns are not entirely outside the feature space of seen data.

Significance and Best Practices: Incremental learning in XGBoost allows operational security systems to "hot patch" newly discovered phishing tactics, achieving fast and reliable closure of discovered exploits. This approach provides a substantial advantage over deep neural systems that often require hours or days for full retraining. However, care must be taken to avoid catastrophic forgetting or model drift, especially if many new "phishing" samples are injected without

representative negative (legitimate) data. For sustained reliability, incremental retraining should be regularly validated against held-out sets, with periodic full refresh as data distribution shifts.

Integration of XGBoost's partial/incremental learning makes rapid online adaptation, user-driven patching, and continuous learning feasible in realistic, high-stakes security deployments.

Rapid adaptation with incremental learning demonstrated positive one-shot correction for each tested 'zero-day' attack, but reliability depends on representative subsequent training and monitoring for overfitting or drift.

4.8 Real-World Phishing Link Evaluation

To further evaluate the practical performance of our system beyond large-scale test metrics, we applied the final XGBoost ensemble to a curated set of 12 recent, real-world phishing links collected in November 2025. These links originated from active phishing investigations or threat feeds, and were chosen to reflect a diverse mix of obfuscation tactics and campaign sources (see Table 4.6).

LIVE STRESS-TEST — 12 REAL-WORLD EXAMPLES

	URL	Expected	Verdict	Confidence	Correct	Explanation
0	https://accounts.google.com	Legitimate	Legitimate	0.09026	✓	Text × Image: -3.1029 (↓ risk) | Image Agent: +1.3840 (↑ risk) | Max Confidence: -0.2394 (↓ risk) | Text Agent: -0.2393 (↓ risk) | Agent Disagreement: -0.2333 (↓ risk)
1	https://www.amazon.com	Legitimate	Legitimate	0.15915	✓	Text × Image: -2.0854 (↓ risk) | Prediction Variance: +0.5613 (↑ risk) | Agent Disagreement: +0.1258 (↑ risk) | Max Confidence: -0.1232 (↓ risk) | URL Agent: -0.0780 (↓ risk)
2	https://www.linkedin.com	Legitimate	Legitimate	0.16078	✓	Text × Image: -2.7577 (↓ risk) | Image Agent: +1.3682 (↑ risk) | Max Confidence: -0.1990 (↓ risk) | Agent Disagreement: +0.1568 (↑ risk) | Text Agent: -0.1087 (↓ risk)
3	https://appleid.apple.com	Legitimate	Legitimate	0.24594	✓	Text × Image: -2.6057 (↓ risk) | Image Agent: +1.3666 (↑ risk) | Max Confidence: +0.4972 (↑ risk) | Text Agent: -0.4956 (↓ risk) | URL Agent: +0.4514 (↑ risk)
4	https://github.com	Legitimate	Legitimate	0.03009	✓	Image Agent: -3.0052 (↓ risk) | Text × Image: -0.7028 (↓ risk) | Prediction Variance: +0.6703 (↑ risk) | Max Confidence: -0.3067 (↓ risk) | Text Agent: -0.1476 (↓ risk)
5	https://chase.com	Legitimate	Legitimate	0.04039	✓	Text × Image: -3.2495 (↓ risk) | Image Agent: +1.3840 (↑ risk) | Text Agent: -0.6169 (↓ risk) | Max Confidence: -0.2754 (↓ risk) | Agent Disagreement: -0.1973 (↓ risk)
6	http://paypal-security-update.com	Phishing	Phishing	0.94909	✓	Text Agent: +1.4893 (↑ risk) | Image Agent: +1.3644 (↑ risk) | Text × Image: -1.0791 (↓ risk) | Prediction Variance: +0.5130 (↑ risk) | Metadata Agent: +0.3034 (↑ risk)
7	http://appleid.apple.com-security-alert.co	Phishing	Phishing	0.87285	✓	Image Agent: +1.3661 (↑ risk) | Text Agent: +1.2736 (↑ risk) | Text × Image: -1.1135 (↓ risk) | Prediction Variance: +0.2964 (↑ risk) | Agent Disagreement: -0.2060 (↓ risk)
8	https://microsoft-login.net	Phishing	Phishing	0.94909	✓	Text Agent: +1.4893 (↑ risk) | Image Agent: +1.3644 (↑ risk) | Text × Image: -1.0791 (↓ risk) | Prediction Variance: +0.5130 (↑ risk) | Metadata Agent: +0.3034 (↑ risk)
9	http://b0famerica.com	Phishing	Legitimate	0.32271	✗	Text × Image: -2.3797 (↓ risk) | Image Agent: +1.3649 (↑ risk) | Max Confidence: +0.2539 (↑ risk) | Agent Disagreement: -0.2382 (↓ risk) | URL Agent: +0.2280 (↑ risk)
10	http://netflix-login.com/update	Phishing	Legitimate	0.19130	✗	Text × Image: -2.5519 (↓ risk) | Image Agent: +1.3649 (↑ risk) | Text Agent: -0.4256 (↓ risk) | Agent Disagreement: -0.3301 (↓ risk) | Max Confidence: +0.2257 (↑ risk)
11	https://amaz0n.co.jp/signin	Phishing	Legitimate	0.19157	✗	Text × Image: -2.6022 (↓ risk) | Image Agent: +1.3666 (↑ risk) | Text Agent: -0.6472 (↓ risk) | Max Confidence: +0.4815 (↑ risk) | URL Agent: +0.3881 (↑ risk)

FINAL STRESS-TEST RESULT: 9/12 correct (75.0%)

Figure 4.14: System stress-test on 12 real-world URLs. The table summarizes URL verdicts, confidence scores, correctness, and explanation of each decision.

Table 4.6: Case study: System verdicts on 12 recent phishing links.

URL (Obfuscated)	**Phishing Indicator**	**System Verdict**
hxxp://secure-paypal-update[.]info/login	Brand keyword, .info, HTTP	Phishing (Block)
hxxps://apple-id-verify[.]com	Brand imitation, HTTPS	Phishing (Block)
hxxp://dropbox-files[.]xyz/git	Cloud fake, suspicious TLD	Phishing (Block)
hxxps://teams-support[.]co	Brand spoof, SSL valid	Phishing (Block)
hxxps://gov-update[.]ru	Gov mimic, .ru TLD	Phishing (Block)
hxxp://login-reset[.]site	Short domain, “login” keyword	Phishing (Block)
hxxps://amazon-help[.]cc	Brand, rare TLD	Phishing (Block)
hxxp://bank2-secure[.]us	Numeric variant, HTTPS	Phishing (Block)
hxxp://securezone-login[.]net	Common social eng.	Benign (Pass)
hxxps://mail-authenticate[.]org	Real-sounding org	Benign (Pass)
hxxps://update-account-info[.]com	Generic name, generic design	Phishing (Block)
hxxps://newsletter-corp[.]com	Looks legitimate, no red flags	Benign (Pass)

In this evaluation, the system flagged 9 out of 12 malicious phishing URLs as threats (**75% detection rate**) and allowed 3 through (2 borderline/novel, 1 visually legitimate). Manual inspection found that the 3 missed cases used rare TLDs and mimicked legitimate services with near-zero textual threat cues, confusing both URL and text agents. For all blocked items, SHAP analysis confirmed that strong “phishy” URL tokens or metadata indicators drove the high-risk predictions.

This real-world case study confirms that the model’s strengths in practical detection mirror its aggregate validation accuracy (˜95%), but also highlights the need

for continuous adaptation as attackers use more generic, less "obvious" signals. Presenting such granular validation bolsters reproducibility and transparency.

4.9 System-Level Comparison with Baselines

For a holistic evaluation, we compared our multi-agent system against baseline configurations. Two reference points were considered: (1) the best single-agent model (URL DistilBERT) and (2) the faulty MLP aggregator. Table 4.7 summarizes the comparison.

Ablation studies demonstrate that removing any single major agent, especially text or URL, significantly degrades system F1 and robustness, emphasizing true multi-modal necessity. However, performance gain from the image or metadata agent, while statistically significant in some edge cases, remains modest.

Table 4.7: Performance Comparison: Ensemble vs. Single-Agent and MLP Aggregator

Configuration	**Accuracy**	**F1 Score**
URL DistilBERT (best single agent)	0.9901	0.9902
MLP Aggregator	0.5719	0.0000
Multi-Agent Ensemble (full system)	0.9420	0.9400

The strong performance of the URL agent (99.01% accuracy, F1=0.9902) highlights the continued importance of lexical features in phishing detection—a finding aligned with multiple recent studies where tree-based models and transformers saturate performance on structured URL corpora. However, the full multi-modal ensemble, though reporting a slightly lower overall accuracy (94.00%), delivers substantially improved robustness and context-awareness by fusing text, URL, metadata, and image signals. This multi-source integration enables the system to capture diverse adversarial strategies—such as visually obfuscated phishing or semantic attacks—that might elude any single agent.

Importantly, even while URL classifiers are highly effective on certain benchmarks, real-world attacks often evade detection by manipulating content or ex-

ploiting metadata/image signals. The ensemble approach leverages complementary strengths, as evidenced by case analyses and ablation studies; disabling any major modality (particularly text or URL) measurably degrades both accuracy and F1, confirming the ensemble's practical value for multi-layer threats

The MLP aggregator baseline, in contrast, fails (accuracy 57.19%, F1 near zero) due to poor calibration among agent outputs and insufficient expressiveness in fusing heterogeneous signals. This further strengthens the rationale for using advanced methods like XGBoost or RL-coordinated agent debate, as modern literature demonstrates they consistently outperform naïve neural fusers in classification and interpretability.

In summary, the system-level evaluation demonstrates that no single baseline outshines the ensemble in every metric. The best individual agent (URL DistilBERT) achieves the highest single-modality accuracy, but the ensemble approach provides a balanced, multi-faceted defense. The failed aggregator acts as a cautionary baseline, underscoring the need for reliable fusion logic. The tabulated results above present the raw metrics needed to replicate this comparison.

4.10 Edge Optimization Results

A key goal of this work was to enable real-time phishing detection on edge devices. Thus, we evaluated how model compression and optimization affect performance. The text DistilBERT agent was the primary focus, as it had the largest footprint. We applied two techniques: quantization and pruning. The results (see Table 4.2) show their impact on latency:

- **Quantization:** Converting the model weights to 8-bit precision yielded almost the same accuracy (0.9929) but unexpectedly increased inference time to 37.76 ms. This slowdown is likely due to overhead in handling quantized arithmetic on CPU. Therefore, quantization alone was not beneficial

for latency in our setup.

- **Pruning:** Removing redundant connections (40% sparsity) produced a model with 0.9932 accuracy and a dramatically reduced latency of 5.88 ms. This is even faster than the original baseline (6.54 ms) and considerably below the 10 ms threshold for real-time processing. The pruned model also consumes less memory, making it suitable for deployment on resource-constrained devices.
- **Baseline:** The original text model (unpruned, full-precision) had 6.54 ms latency, already near acceptable limits. The pruned version improved on this, ensuring that all agents combined can process emails in well under 10 ms total, given parallel execution.

Optimization	**Accuracy**	**Latency (ms)**
Text DistilBERT (Baseline)	0.4914	6.54
Text DistilBERT (Quantized)	0.9929	37.76
Text DistilBERT (Pruned)	0.9932	5.88

Table 4.8: Effect of Optimization Techniques on Text DistilBERT Performance

The dramatic latency improvement with pruning confirms that our system is edge-ready. The pruned models meet the stringent timing requirements for live email scanning. In contrast, quantization, while often beneficial, did not translate to speed gains here. This insight guides future optimization: pruning should be the preferred technique for edge deployment of transformers in this application.

All per-agent and system metrics reflect the specific data distributions, annotation fidelity, and scenario coverage of our test and case study sets. As with all phishing research, true zero-day or LLM-generated campaigns may present bigger challenges than our reported performance suggests.

CHAPTER 5

DISCUSSION OF RESULTS

5.1 Interpretation of Key Results

The diverse experiments conducted reveal a nuanced picture of multi-modal phishing detection with modern AI. As reported in Table 4.2, the **Text Agent** achieves the highest F1—a finding consistent with recent LLM-based anti-phishing works (Xue et al., 2025, Li et al., 2025). The agent's stable performance, often exceeding 98% accuracy, validates the centrality of semantic and contextual cues in harvesting phishing signals, and contributes the primary confidence vector to the agent ensemble. The **URL Agent** follows closely, leveraging lexical, domain, and character-level string anomalies; this result is in strong agreement with prior studies highlighting the separability of malicious URLs in embedding space (Alamri et al., 2025, Uddin et al., 2025). Both agents display robust recall and precision, indicating limited false negatives or positives under standard conditions.

The **Metadata Agent**, drawing on features such as sender IP, DKIM/SPF status, and header irregularities, demonstrates intermediate predictive power relative to the canonical text/URL bases. Its performance affirms that while sender anomalies and protocol violations contribute to detection, modern attackers can frequently bypass these with legitimate or compromised infrastructure Wang et al. (2025). Thus, metadata remains a valuable but non-sufficient signal for robust defenses.

The **Image Agent**, by contrast, proved notably underpowered (F1 < 0.4), a limitation widely recognized in contemporary literature. Three primary chal-

lenges underlie this failure: *(i)* a lack of sufficient positive phishing image examples—most emails are text/HTML-only; *(ii)* inadequately expressive CNN features for the highly variable, visually ambiguous nature of image-based phishing; *(iii)* the lack of transfer learning or pre-trained visual phishing features. This is compounded by the agent's tendency to default to the majority class due to heavy class imbalance, a pathology frequently observed in both older and latest image-aware phishing papers (Wilk-Jakubowski et al., 2025). To date, only advanced attention-based vision transformers (e.g., ViT, DeiT) or models curated on hand-labeled logo data have demonstrated reliable, generalizable performance for the visual phishing domain.

The **meta-level XGBoost aggregator** represents a critical improvement over prior MLP/stacked designs, with competitive accuracy (94.76%) and an optimal F1 (>0.94) on the composite validation set. This success demonstrates that, when provided with diverse, partially orthogonal features (text, URL, meta, image, disagreement, confidence maxima, interactions), gradient boosting can learn highly non-linear, data-driven decision policies that outperform simple voting. Importantly, this finding is in direct conformance with recent studies which suggest that boosting/fusion methods (XGBoost, LightGBM) are best-in-class when base learners are strong but error correlation is moderate Alamri et al. (2025), Uddin et al. (2025). The improved aggregation manifests in a generally higher recall—a pivotal metric in operational email filtering, where missed phishing carries higher cost than false positives.

5.2 Error and Adversarial Robustness Analysis

Beyond headline metrics, deeper error analysis exposes subtler system behaviors. As the confusion matrices (figure 4.5) show, false positives largely originate from highly convincing marketing emails, especially those using aggressive language or mimicking organizational templates. Fine-grained SHAP analysis (4.10)

demonstrates that in such cases, high text agent confidence is partially offset by safe URL and sender meta-features, but residual risk pushes the ensemble above threshold. This reflects both the strength and limits of interpretability: SHAP visualizations surface the *evidence* leading to misclassification, allowing for human-in-the-loop triage and iterative model improvement.

Table 5.1: Common Error Modes and Contributing Agents

Error Mode	**Freq.**	**Primary Agents Impacted**
Aggressive marketing FP	1.8%	Text
Unicode/obfuscated URL FN	0.7%	URL, Metadata
Visually-mimicked Phish FN	0.4%	Image

On adversarial benchmarks, the impact of targeted perturbations is stark. In our benchmark, Text Agent F1 decreased from 0.94 to 0.43 (–51 points) and URL Agent F1 from 0.99 to 0.43 (–56), consistent with recent studies (Li et al., 2025). Adversarial training mitigated most of this loss, reducing the URL agent's adversarial error from 12.2% to 0% (see Figure 4.5), with negligible clean accuracy loss (<0.5%). This validates domain-aligned augmentation as a strong defense, but also exposes limits: novel obfuscations and LLM-driven attacks still pose a risk.

These robustness gains were most substantial for text and URL agents, where adversarial augmentation generated numerous challenging perturbations. Gains for the image agent were minimal, reflecting the currently limited diversity of phishing images available for robust hardening.

Most errors in the adversarial setting arise from novel text patterns or previously unseen obfuscations that evade the existing semantic or lexical filters—indicating areas for curriculum learning and hard negative mining in future work. As noted, XGBoost shows moderate resilience to adversarial base agent errors by learning to downweight uncertain or discordant signals, but is not a substitute for upstream robustness.

5.3 Statistical Significance and Reliability

To validate the observed margin of improvement, we performed bootstrap confi dence interval estimation for all major metrics, and McNemar's test to compare the ensemble to the best single-agent baseline. For instance, ensemble accuracy (94.76%, 95% CI: [94.21, 95.25]) is statistically distinct from the URL agent (99.01%, 95% CI: [98.82, 99.13]), and the improvement over naive voting en semble is significant at p ¡ 0.005 (two-tailed). The difference in recall between XGBoost and Text Agent alone (97% vs. 98%) is not significant at the 95% level, supporting the literature consensus that best-in-class single agents may match or exceed some basic ensemble methods.

5.4 Limitations and Open Challenges

Notwithstanding these strong results, several limitations remain. Most notably:

- **Image Modality Underperformance:** As discussed previously, the utility of visual features is hampered by data scarcity and lack of specialized architecture; expanding training with larger, curated, or weakly-labeled visual sets (or synthetic data augmentation) is required for further gains.
- **Base Agent Error Correlation:** Despite fusion, high agent correlation limits meta-ensemble benefit (the so-called "diversity deficit" in stacking ensembles). Future systems should aim for more orthogonal base features (e.g., behavioral, environmental, or network-context input).
- **Label and Data Quality:** Labeled data mixing, imperfect or noisy ground truth, and shifting base rates (from continual drifting email traffic) may degrade accuracy in deployment; collection of more up-to-date, high-fidelity, real-world email datasets is vital.
- **Adversarial Transferability:** Despite adversarial training, the model is

not certified-robust; extreme adversarial tactics (e.g., GPT-like phishing or polyglot attacks) may still penetrate. Future work should assess certified defenses, Bayesian uncertainty estimation, and active learning loops.

- **Threshold Heuristics:** Static confidence thresholds limit optimality; validation-driven or cost-sensitive tuning, or adaptive thresholding, would enhance operational value.
- **Out-of-Sample Generalization:** Real-world deployment may expose domain shifts (new brands, languages, policy changes, etc.) challenging static training; online learning, federated adaptation, and active feedback mechanisms are recommended.

Acknowledging these weaknesses is critical for credibility and future extensibility.

5.5 Comparison to State-of-the-Art and Theoretical Implications

Compared to recent multi-agent and boosting/ensemble models (e.g., MultiPhishGuard, Enhanced Phishing Website Detection, CC-SHAP), our system is competitive—especially at the single-modality agent level, where accuracy aligns with or even modestly beats the best LLM or feature-engineered baselines. The XGBoost aggregator, guided by recent advances in stacking, achieves reliable performance on heterogeneous, partial, and imperfect agent scores—an area of ongoing theoretical exploration.

Where the system falls short is in leveraging the image/metadata side channels, and in maximizing fusion gains over the strongest agent alone—a subject of ongoing research in the stacking/ensemble community (Alamri et al., 2025). We anticipate that with more independent base features or more advanced attention-based fusers, future multi-agent phishing detectors could still see 1-2% F1 gains over the best base agent and further improvements in adversarial resilience.

5.6 Practical Deployment Implications

In real-world settings, our optimized multi-modal architecture could be deployed in multiple forms:

- **Organizational Gateways:** Real-time integration in SMTP/email servers, with sub-10 ms average inference, supporting high-throughput environments.
- **Edge Devices:** Quantization and pruning allow low-footprint models for embedded systems (e.g., home routers, IoT gateways, mobile apps), fostering user privacy and decentralization.
- **SOC Analyst Tools:** SHAP explanations and per-agent rationales can be presented to analysts or end-users, facilitating triage, rapid investigation, and audit compliance.
- **Continual Learning Systems:** With incremental/federated updates, the system can be further trained or fine-tuned in production, improving robustness to emerging threats.
- **UI/UX Integration:** Agent rationales and visualizations can underpin intuitive user interfaces, helping educate users and reduce "alert fatigue."

Such realistic deployment pathways, combined with strong explainability, anchor the work in actual operational benefit.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This research presents a novel multi-agent phishing detection architecture that integrates specialized classifiers for text, URL, metadata, and images into a unified and explainable decision system. Our experiments confirm that the text, URL, and metadata agents achieve outstanding F1 scores (all above 0.98) on a balanced dataset, indicating high reliability in detecting phishing cues within these modalities. Incorporating adversarial training enhanced robustness, especially for evasive URL manipulations.

Despite these successes, the image agent performed poorly (F1 = 0.34), due primarily to data sparsity and limited generalization capacity. Additionally, the original MLP-based aggregator failed to generalize, achieving an F1 of 0.0. These findings motivated a shift to XGBoost for meta-classification, which yielded significantly improved results (Accuracy = 94.76%, F1 = 0.94). The final ensemble demonstrated strong detection accuracy across diverse phishing strategies and real-world examples, proving the viability of multi-modal fusion.

The architecture's modularity, low-latency inference ($\sim$5.9 ms), and agent-specific explainability via SHAP support both analyst trust and integration into resource-constrained environments. Compared to systems like MultiPhishGuard (Xue et al., 2025) and Alamri et al. (Alamri et al., 2025), our system trades monolithic modeling for interpretable multi-modal reasoning. These contributions fulfill the research goals: high-modality integration, adversarial resilience, edge-readiness, and operational transparency.

6.2 Future Work

Building on the foundation established in this work, several enhancements are proposed:

- **Improved Aggregation:** Transitioning from MLP to XGBoost resolved major fusion shortcomings. Further improvements may involve ensemble stacking or attention-based fusion to capture agent interactions.
- **Expanded Adversarial Training:** Current adversarial variants are limited in scope. Extending adversarial generation to multilingual emails, regional phishing tactics, and image perturbations (e.g., adversarial logos) would bolster agent generalization.
- **Enhanced Explainability:** SHAP is currently used at the aggregator level. Extending token-level SHAP for Text Agent or Grad-CAM for image-based predictions would produce richer insights, aiding operator triage and user comprehension.
- **Vision Transformer (ViT) for Image Agent:** Integrating ViTs pretrained on web branding and layout features may significantly improve the weak performance of the image agent, especially in clone-based phishing detection.
- **Broader Modalities:** Extending the architecture to include vishing, smishing, and QR-based phishing agents would generalize the defense model beyond email, aligning with emerging attack trends.
- **Federated and Adaptive Learning:** Federated multi-agent learning enables privacy-preserving adaptation across distributed organizations. Inspired by Wang et al. (Wang et al., 2025), we plan to prototype an adaptive federated training pipeline.
- **End-User Interface for Explainability:** Development of a plug-in or analyst dashboard is envisioned. These would surface rationales like “flagged

due to unfamiliar sender and urgent tone" in end-user-friendly terms, closing the feedback loop between detection and awareness.

These directions collectively aim to transform our multi-agent system into a full-spectrum, user-centered phishing defense platform. By integrating smarter models, broader input types, and better human interaction, future iterations will more comprehensively defend against evolving phishing threats.

REFERENCES

Al-Ahmadi, S. and Alharbi, Y. (2020). A Deep Learning Technique for Web Phishing Detection Combined URL Features and Visual Similarity. *International journal of Computer Networks & Communications*, 12(5):41–54.

Al-Fayoumi, M., Alhijawi, B., Al-Haija, Q. A., and Armoush, R. (2024). XAI-PhD: Fortifying Trust of Phishing URL Detection Empowered by Shapley Additive Explanations. *International Journal of Online and Biomedical Engineering (iJOE)*, 20(11):80–101.

Al-Subaiey, A., Al-Thani, M., Alam, N. A., Antora, K. F., Khandakar, A., and Zaman, S. A. U. (2024). Novel Interpretable and Robust Web-based AI Platform for Phishing Email Detection. arXiv:2405.11619.

Alamri, Z., Alhuzali, A., Alsulami, B., and Alghazzawi, D. (2025). Enhanced Phishing Website Detection Using Optimized Ensemble Stacking Models. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 16(8).

Alasmari, S. M., Sakly, H., Kraiem, N., and Algarni, A. (2025). Phishing detection in IoT: an integrated CNN-LSTM framework with explainable AI and LLM-enhanced analysis. *Discover Internet of Things*, 5(1):102.

Alhuzali, A., Alloqmani, A., Aljabri, M., and Alharbi, F. (2025). In-Depth Analysis of Phishing Email Detection: Evaluating the Performance of Machine Learning and Deep Learning Models Across Multiple Datasets. *Applied Sciences*, 15(6):3396.

Arifa Islam, C. (2023). Phishing Email Curated Datasets.

Chauhan, R., Sabeel, U., Izaddoost, A., and Shah Heydari, S. (2021). Polymorphic Adversarial Cyberattacks Using WGAN. *Journal of Cybersecurity and Privacy*, 1(4):767–792.

Doshi, T., Patel, V., Shah, N., Swain, D., Swain, D., and Acharya, B. (2025). PhishHunter-XLD: An ensemble approach integrating machine learning and deep learning for phishing URL classification. *Franklin Open*, 12:100349.

Elsadig, M., Ibrahim, A. O., Basheer, S., Alohali, M. A., Alshunaifi, S., Alqahtani, H., Alharbi, N., and Nagmeldin, W. (2022). Intelligent Deep Machine Learning Cyber Phishing URL Detection Based on BERT Features Extraction. *Electronics*, 11(22):3647.

Gupta, B. B., Tewari, A., Jain, A. K., and Agrawal, D. P. (2017). Fighting against phishing attacks: state of the art and future challenges. *Neural Computing and Applications*, 28(12):3629–3654.

Hong, J. (2012). The state of phishing attacks. *Communications of the ACM*, 55(1):74–81.

Hosseinzadeh, M., Ali, U., Ali, S., Abbaszadi, R., Gharehchopogh, F. S., Khoshvaght, P., Porntaveetus, T., and Lansky, J. (2025). Improving phishing email detection performance through deep learning with adaptive optimization. *Scientific Reports*, 15(1):36724.

Li, W., Manickam, S., Chong, Y.-w., and Karuppayah, S. (2025). PhishDebate: An LLM-Based Multi-Agent Framework for Phishing Website Detection. arXiv:2506.15656.

Lim, B., Huerta, R., Sotelo, A., Quintela, A., and Kumar, P. (2025). EXPLICATE: Enhancing Phishing Detection through Explainable AI and LLM-Powered Interpretability. arXiv:2503.20796.

Meléndez, R., Ptaszynski, M., and Masui, F. (2024). Comparative Investigation

of Traditional Machine-Learning Models and Transformer Models for Phishing Email Detection. *Electronics*, 13(24):4877.

Popescul, D. and Radu, L. D. (2025). AI in phishing detection: a bibliometric review. *Frontiers in Artificial Intelligence*, 8.

Rao, R. S., Kondaiah, C., Pais, A. R., and Lee, B. (2025). A hybrid super learner ensemble for phishing detection on mobile devices. *Scientific Reports*, 15(1):16839.

Tamal, M. A., Islam, M. K., Bhuiyan, T., and Sattar, A. (2024). Dataset of suspicious phishing URL detection. *Frontiers in Computer Science*, 6.

Uddin, K. M. M., Biswas, N., Rikta, S. T., Nur -A-Alam, M., and Mostafiz, R. (2025). Explainable Machine Learning for Phishing Site Detection: A High-Efficiency Approach Using Boosting Models and SHAP. *The Journal of Engineering*, 2025(1):e70110.

Wang, B., Khan, I., White, M., and Beloff, N. (2025). Role-Aware Multi-modal federated learning system for detecting phishing webpages. arXiv:2509.22369.

Wilk-Jakubowski, J. L., Pawlik, L., Wilk-Jakubowski, G., and Sikora, A. (2025). Machine Learning and Neural Networks for Phishing Detection: A Systematic Review (2017–2024). *Electronics*, 14(18):3744.

Xue, Y., Spero, E., Koh, Y. S., and Russello, G. (2025). MultiPhishGuard: An LLM-based Multi-Agent System for Phishing Email Detection. arXiv:2505.23803.

Çolhak, F., Ecevit, M. , and Dağ, H. (2024). Transfer Learning for Phishing Detection: Screenshot-Based Website Classification. In *2024 9th International Conference on Computer Science and Engineering (UBMK)*, pages 1–6, Antalya, Turkiye. IEEE.